개발 문서

온기(溫氣): 아이의 모든 순간을 함께할 부모의 목소리 유산함

프로젝트 구분	2026 학년도 2 학기 모바일시스템응용프로젝트 (7 분반 5 팀)
프로젝트명	온기(溫氣)
팀명	AI 큐
작성일	2026-11-12
문서 버전	v1.2 (최종 시연본)
지도교수	정병덕 교수님
산업체 멘토	(주)마음 AI 김민선, 한하평 매니저
구성원	공지현(팀장), 박석준, 황정범, 서유정

목차

문서 구성 4
현재 상태 4
표기 방법 4
역할 분담 4
한눈에 보는 온기 5
온기는 무엇을 하나 5
아이가 말을 걸면 생기는 일 5
어떤 순간인지 찾는 세 단계 6
자주 나오는 말 쉽게 풀기 6
핵심 설계 결정 8
1. 요구사항 정의서 9
1.1 문서 개요 9
1.2 프로젝트 개요 9
1.3 기능적 요구사항 10
1.4 비기능적 요구사항 16
1.5 환경 요구사항 18
1.6 제약사항 18
2. 아키텍처 설계서 20
2.1 문서 개요 20
2.2 시스템 개요 20
2.3 하드웨어 아키텍처 20
2.4 소프트웨어 아키텍처 21
2.5 데이터 아키텍처 22
2.6 주요 기술 스택 23
3. 모델 설계 및 최적화 문서 24
3.1 문서 개요 24
3.2 모델 선정 및 개요 24
3.3 데이터셋 구성 26
3.4 모델 학습 및 평가 전략 27
3.5 모델 경량화 및 최적화 방안 28
3.6 추론 엔진 및 런타임 환경 28
4. API 및 인터페이스 정의서 30
4.1 문서 개요 30
4.2 인터페이스 개요 30
4.3 API 목록 및 상세 정의 30
4.4 통신 프로토콜 및 형식 34
4.5 인증 및 보안 34
5. 테스트 계획서 및 결과 보고서 36

5.1 테스트 계획서 36
5.2 테스트 결과 보고서 38
6. 배포 및 통합 가이드 42
6.1 문서 개요 42
6.2 배포 환경 설정 42
6.3 모델 및 추론 엔진 설치 43
6.4 온디바이스 시스템 통합 방법 43
6.5 에러 처리 및 트러블슈팅 가이드 45
6.6 업데이트 및 유지보수 전략 46
부록 A. 상황 12 개 대응표 48

문서 구성

본 문서는 온기의 요구사항, 아키텍처, 모델, 인터페이스, 테스트, 배포를 여섯 개 장으로 다룬다.

장	문서	다루는 내용
개요	한눈에 보는 온기	개발을 전공하지 않아도 그림으로 이해하는 전체 구조와 동작
요약	핵심 설계 결정	무엇을 왜 골랐는가
1	요구사항 정의서	무엇을, 누구를 위해, 어느 수준으로 만드는가
2	아키텍처 설계서	앱, 서버, 디바이스가 어떤 구조로 맞물리는가
3	모델 설계 및 최적화 문서	어떤 모델과 데이터를 쓰고 어떻게 가볍게 만드는가
4	API 및 인터페이스 정의서	구성 요소가 서로 무엇을 주고받는가
5	테스트 계획서 및 결과 보고서	무엇을 어떻게 검증하고 결과는 어떠한가
6	배포 및 통합 가이드	디바이스에 어떻게 올리고 연결하는가
부록 A	상황 12 개 대응표	화면, 데이터, 모델이 함께 쓰는 상황 기준표

현재 상태

온기는 부모 앱, 서버, 디바이스(QCS6490 보드), 자녀 앱의 네 부분으로 이루어지며, 부모가 남긴 메시지가 서버를 거쳐 디바이스에 저장되고 아이의 말에 맞춰 재생되는 한 사이클이 실제 마이크와 스피커로 동작한다. 아이 말의 인식과 판단은 모두 디바이스 안에서 이루어지고, 자녀 이름 공유, 스피커 연결 상태, 메시지 작성 유도, 기록 지우기 기능까지 구현했다. 학습한 LLM 은 (주)마음 AI 의 양자화를 거쳐 디바이스에서 실행하며, 최종평가(11 월 26 일)에서 실제 장비로 시연한다.

1·2 장은 설계를, 3~6 장은 모델, 인터페이스, 시험 결과, 배포 방법을 담았다. 개발을 전공하지 않은 독자는 "한눈에 보는 온기"부터 읽으면 된다.

표기 방법

각 기능과 기술에는 이번 학기에 만든 것인지(구현), 구조만 정해 둔 것인지(설계)를 표시했다. 아직 측정하지 않은 값은 목표값으로 적었다.

역할 분담

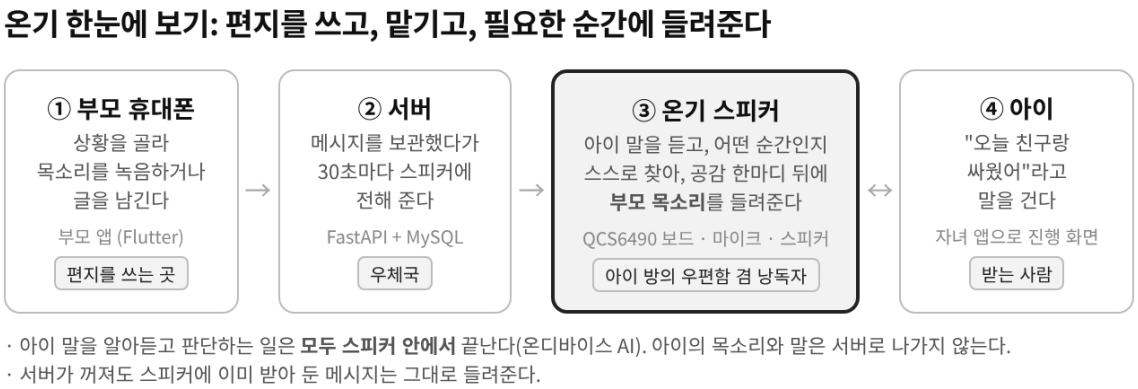
구분	이름	개발 역할
팀장	공지현	디바이스: 보드 안 듣기·판단·재생 흐름, 앱 안 모델 실행(STT·TTS·임베딩·LLM), 오디오, 서버 동기화, 앱 신호 규격 / LLM 학습 데이터·학습·병합 / 일정 관리, 개발 문서 통합
팀원	박석준	AI·기자재: 데이터 검수, 보드·사운드카드·공유기 관리, LLM 발표 자료
팀원	황정범	서버·DB: FastAPI·MySQL, API 14 개, 배포, 테스트 결과표
팀원	서유정	앱: Flutter 부모·자녀 화면 28 장, 녹음, 서버·디바이스 연결, 시연 영상

한눈에 보는 온기

이 장은 개발을 전공하지 않은 사람도 온기가 어떻게 동작하는지 이해할 수 있도록 그림과 쉬운 말로 설명한다. 정확한 설계와 측정값은 1~6 장에 있고, 각 그림 아래에 해당 절을 적었다.

온기는 무엇을 하나

온기는 부모가 미리 남긴 목소리를, 아이에게 그 말이 필요한 순간에 들려주는 스피커다. 부모는 휴대폰으로 "시험을 망친 날", "친구와 싸운 날" 같은 상황을 골라 녹음이나 글을 남긴다. 아이가 스피커에 "오늘 친구랑 싸웠어"라고 말하면, 스피커가 어떤 순간인지 스스로 알아듣고 짧은 공감 한마디 뒤에 그 상황에 남겨 둔 부모의 목소리를 들려준다.



<그림 0-1> 온기 한눈에 보기: 부모 휴대폰(편지 쓰기) → 서버(우체국) → 온기 스피커(아이 방의 우편함 겸 낭독자)

- 부모 휴대폰(부모 앱): 상황을 골라 메시지를 남기고, 아이가 언제 어떤 메시지를 들었는지 본다.
- 서버: 부모가 남긴 메시지를 보관했다가 스피커가 30 초마다 물어볼 때 새 메시지를 건넨다. 편지를 맡아 두는 우체국과 같다.
- 온기 스피커(QCS6490 보드): 마이크로 아이 말을 듣고, 판단하고, 공감 한마디와 부모 메시지를 소리로 들려준다. 판단에 쓰는 AI 모델 네 개가 모두 이 안에 들어 있다.
- 자녀 휴대폰(자녀 앱): 말하기 버튼과 진행 화면(듣는 중, 생각하는 중, 공감, 메시지)을 보여 주고, 들은 메시지를 온기기록에 모은다.

아이가 말을 걸면 생기는 일

아이의 말이 끝나고 첫 소리가 나기까지 약 1.6~1.8 초가 걸린다(목표 3 초). 이 과정은 모두 스피커 안에서 끝나고, 서버로는 "몇 시에 어떤 메시지를 들었는지"만 올라간다.

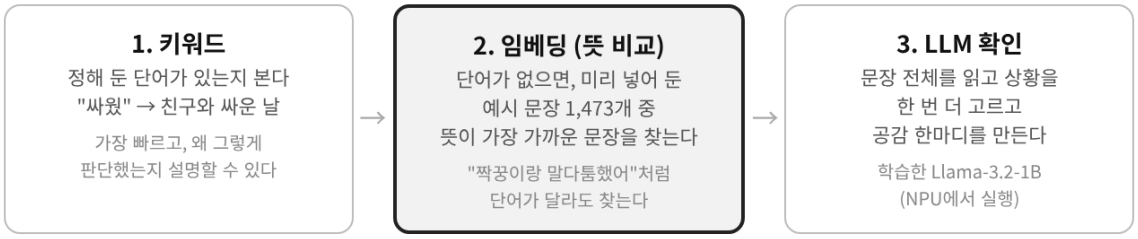


<그림 0-2> 아이가 말을 걸면 스피커 안에서 생기는 일(시간은 11/9 실제 마이크 측정, 2.4 절·5.2.3 절)

이렇게 기기 안에서 처리하는 방식을 온디바이스 AI 라고 한다. 온기가 이 방식을 고른 이유는 세 가지다. 첫째, 아이의 목소리와 속마음이 담긴 말이 집 밖으로 나가지 않는다. 둘째, 인터넷이나 서버가 끊겨도 이미 받아 둔 메시지는 들려준다. 셋째, 말할 때마다 외부 AI 를 부르는 비용이 들지 않는다.

어떤 순간인지 찾는 세 단계

온기에서 가장 중요한 일은 아이의 말을 듣고 "지금이 어떤 순간인지"를 맞히는 것이다. 엉뚱한 순간에 부모 메시지가 나오면 오히려 상처가 될 수 있으므로, 확실한 방법부터 차례로 쓰고 자신이 없으면 메시지를 틀지 않는다.



<그림 0-3> 상황을 찾는 세 단계: 키워드 → 임베딩(뜻 비교) → LLM 확인(3.2 절)

세 방법은 서로의 약점을 메운다. 키워드는 빠르고 틀리지 않지만 정해 둔 단어가 없으면 놓친다. 임베딩은 단어가 달라도 뜻이 비슷하면 찾지만 짧은 일상 대화를 상황으로 착각할 수 있어 기준을 엄격하게 두었다. LLM 은 문장 전체를 읽어 가장 똑똑하지만 1B 크기의 작은 모델이라 가끔 틀리므로, 키워드가 잡은 상황은 LLM 이 달라도 그대로 따른다.

자주 나오는 말 쉽게 풀기

용어	쉬운 설명	온기에서는
받아쓰기(STT)	사람 목소리를 글자로 바꾸는 기술	SenseVoice 모델이 아이 말을 글로 바꾼다 (0.25~0.4 초)

용어	쉬운 설명	온기에서는
음성 합성(TTS)	글자를 사람 목소리처럼 읽어 주는 기술	VITS 모델이 공감 한마디와 글 메시지를 읽는다
키워드	상황마다 정해 둔 핵심 단어	"싸웠", "망쳤" 같은 단어가 있으면 바로 그 상황으로 정한다
임베딩	문장을 숫자 384 개로 바꿔, 뜻이 비슷한 문장끼리 가깝게 놓는 기술	미리 넣어 둔 예시 문장 1,473 개 중 가장 가까운 문장으로 상황을 찾는다(3.2 절)
LLM	많은 글을 읽고 배운 언어 모델	팀이 학습시킨 Llama-3.2-1B 가 상황을 다시 고르고 공감 한마디를 만든다
양자화	모델 속 숫자를 작게 줄여 가볍고 빠르게 만드는 일	(주)마음 AI 가 학습 모델을 보드의 NPU 에서 돌 수 있게 줄인다
NPU	AI 계산에 특화된 칩	LLM 은 NPU 에서, 나머지 모델은 CPU 에서 돈다
동기화	두 곳의 내용을 같게 맞추는 일	스피커가 30 초마다 서버에 새로 생기거나 고쳐지거나 지워진 메시지를 묻는다
WebSocket	두 기기가 연결을 열어 둔 채 바로바로 신호를 주고받는 방식	자녀 앱과 스피커가 "듣는 중", "재생 중" 같은 신호를 주고받는다

핵심 설계 결정

온기의 주요 설계 결정은 다음과 같다. 각 항목의 상세는 위치 열의 절에서 다룬다.

결정 항목	선택	검토한 대안	선택한 이유	위치
상황 수	설계·키워드·임베딩은 12 개 (4 분류 × 3), LLM 학습과 시연은 4 개(3 시험, 4 친구, 10 보고 싶음, 11 자랑)	12 개 전부 학습	학기 안에 시연 품질을 확보하려면 학습 범위를 줄여야 한다(11/5 교수님 피드백). 구조는 12 개 그대로 두어 나중에 학습만 늘리면 된다	1.3, 3.3
상황 판단 구조	키워드 규칙 → 임베딩으로 상황을 정하고, LLM 이 상황을 다시 고름. **판단 규칙 A**: 키워드가 잡은 상황은 LLM 이 달라도 키워드를 따름	두 판단이 같을 때만 재생	1B 모델이 틀려도 시연이 끊기지 않게 한다. 키워드는 학습·평가 데이터에서 틀린 적이 없다. LLM 이 맞으면 학습한 공감 문장을 쓴다	2.4, 3.2
LLM 의 출력	상황·기분 번호 + 공감 한마디 한 문장	번호만 출력	아이 말에 맞는 공감을 만들기 위해서다. 생성 문장은 규칙으로 검사하고, 어기면 미리 쓴 문장으로 바꾼다	3.2
LLM 의 입력	발화 하나(마음 AI 기본 템플릿)	system 지시 + 후보와 발화	마음 AI 의 양자화와 보드 실행 도구를 그대로 쓸 수 있다	3.2
LLM 학습	팀이 Colab 에서 LoRA 로 학습·병합, 양자화만 마음 AI	마음 AI GPU 로 학습	GPU 를 받지 못했다. 학습 결과를 팀이 직접 채점하고 고칠 수 있다	3.4
보드 안 구조	디바이스 앱(Kotlin) 안에서 모델을 직접 실행(Genie, sherpa-onnx, ONNX Runtime)	보드 안 AI 서버 (HTTP)	보드가 Android 라 앱 하나에 묶는 쪽이 단순하고, 보드에 들어 있던 SUDA 앱과 같은 방식이다	2.4
오디오 출력	root 도우미가 tinyplay·tinycap 으로 USB 사운드카드를 직접 사용	앱의 AudioTrack·AudioRecord / 맥에서 재생	이 보드는 USB 오디오 오프로드 때문에 앱 경로로 소리가 나지 않는다. 하드웨어를 직접 쓰면 양방향 모두 된다	2.3
메시지 보관	디바이스가 원본을 보관, 서버는 전달 통로	클라우드에 보관	서버가 사라져도 이미 받은 메시지는 재생되어야 한다	2.2
앱과 디바이스 통신	같은 공유기 안에서 WebSocket 직접 연결	서버 경유 (MQTT) / 블루투스	인터넷 없이 동작하고, 디바이스가 진행 상태를 먼저 알릴 수 있다	4.4
구현 범위	메시지 등록부터 재생까지 한 사이클, 서버는 로컬(공유기) 우선	로그인, 페어링, IoT Core 까지 전부	4 명이 5 주 안에 검증할 수 있는 범위를 우선한다. AWS 배포는 시간이 남으면 한다	1.3

1. 요구사항 정의서

온기가 제공할 기능과 지켜야 할 품질 기준을 정의해, 팀과 지도교수, 산업체 멘토가 같은 기준으로 결과물을 확인할 수 있게 한다.

1.1 문서 개요

본 문서는 온기의 기능적 요구사항과 비기능적 요구사항을 정리한다. 팀원 4 명이 개발 범위를 합의하는 기준으로 쓰고, 지도교수와 (주)마음 AI 멘토가 진행 상황을 확인하는 근거로 쓴다.

적용 대상 시스템은 세 가지다.

- 부모 앱, 자녀 앱(Flutter): 메시지를 남기고, 말을 걸고, 들은 메시지를 다시 듣는 화면
- 온기 디바이스(QCS6490 보드, 마이크, 스피커): 음성 인식, 상황 판단, 공감 한마디와 부모 메시지 재생
- 서버(FastAPI): 메시지 저장과 디바이스 전달

용어 정의(온기에 맞춰 정한 용어만 적는다)

용어	뜻
상황	부모가 메시지를 남기는 단위. 4 개 분류, 12 개로 구성되며 번호는 1~12 다(부록 A)
시연 상황	LLM 을 학습하고 시연하는 4 개 상황. 3 시험을 망친 날, 4 친구와 싸운 날, 10 부모님이 보고 싶을 때, 11 자랑하고 싶은 날
키워드 규칙	받아쓴 문장에 상황별 키워드가 있으면 그 상황으로 정하는 단계. 띄어쓰기는 무시하고 비교한다
막는 말	상황 단어가 있어도 그 상황이 아니라고 보는 표현. 남의 일("싸웠대", "들끼리 다툼")과 다른 뜻("눈싸움")
기준 문장	각 상황을 대표하는 아이의 예시 발화. 임베딩해 디바이스에 넣어 두고, 키워드로 정하지 못한 말과 비교한다. 일반 대화는 '없음(0)'으로 함께 넣는다
임베딩 판단	키워드로 정하지 못한 말을 기준 문장과 비교해, 1 순위가 임계값 이상이고 '없음'보다 충분히 가까울 때만 상황으로 정하는 단계
LLM 확인	LLM 이 발화를 읽고 상황·기분을 고르며 공감 한마디를 만들거나, 일반 대화로 판정하는 단계
판단 규칙 A	키워드가 잡은 상황을 LLM 이 일반 대화·다른 상황으로 보거나 답 형식이 깨지면, 키워드의 상황대로 대체 공감과 부모 메시지를 재생하는 규칙
기분	아이가 말에 드러낸 감정 6 종(1~6). 말하지 않았으면 -1
공감 한마디	부모 메시지 전에 온기가 건네는 짧은 문장. LLM 이 만든 문장(생성 공감)을 규칙으로 검사해 쓰고, 어기면 미리 쓴 문장(대체 공감)을 쓴다
연결 문장	공감 한마디와 부모 메시지 사이에 나오는 정해 둔 문장. 녹음이면 "엄마가 민준이한테 남긴 말이 있어. 들려줄게", 글이면 "…읽어 줄게"
받아주는 말	상황을 정하지 못했거나 일반 대화일 때 부모 메시지 대신 들려주는 고정 문장. 예: "그런 날도 있지, 이야기해 줘서 고마워"

1.2 프로젝트 개요

온기(溫氣)는 부모가 자녀의 인생 순간마다 전하고 싶은 말을 미리 남겨두면, 자녀가 그 순간을 맞았을 때 기기가 알아듣고 부모의 말로 전해주는 온디바이스 음성 유산함이다.

부모를 일찍 잃었거나 장기 별거, 해외 거주, 시설 보호로 부모와 떨어져 지내는 아이는 첫 등교, 친구와의 다툼, 시험 결과 같은 순간을 혼자 맞는다. 기존 방식은 이 문제를 풀지 못한다.

기존 방식	한계	온기의 해결
클라우드 유산 서비스	회사가 문을 닫으면 유산이 통째로 사라진다	디바이스가 메시지 원본을 보관한다. 서버는 전달 통로로만 쓴다
키워드 매칭	정해 둔 단어가 없으면 반응하지 않고, 단어가 다른 말에도 걸리면 엉뚱하게 반응한다	키워드가 없는 말은 임베딩이 보조하고, 막는 말로 다른 뜻을 걸러내며, LLM 이 문장 전체를 읽고 한 번 더 확인한다
녹음 파일 보관	아이가 직접 찾아 들어야 해서 힘든 순간에는 쓰이지 않는다	말만 걸면 맞는 메시지가 재생된다

온디바이스 AI 도입 필요성

1. 통신이 끊겨도 동작한다. 메시지 원본은 디바이스에 보관하므로 서버가 사라져도 이미 받은 메시지는 재생된다.
2. 아이의 감정 발화가 밖으로 나가지 않는다. 인식, 판단, 재생이 모두 기기 안에서 끝난다.
3. AI 추론에 외부 호출 비용이 없다.

서비스 철학

AI 는 아이의 말뜻을 알아듣고 부모의 말로 이어주는 역할을 한다. 전달되는 부모 메시지는 모두 부모가 직접 쓰거나 녹음한 내용이며, AI 는 부모의 말을 지어내지 않는다. 부모 메시지 앞의 공감 한마디는 LLM 이 아이 말에 맞게 만들되, 길이·말투·금지 표현 규칙을 통과한 문장만 내보내고, 어기면 사람이 미리 쓴 문장으로 바꾼다. 공감 한마디는 부모가 아닌 온기의 목소리로 나온다.

부모 메시지는 부모의 녹음으로 남기는 것이 목표다. 이번 학기 시연에서는 부모 메시지도 온기 목소리(TTS)로 읽고, 부모 목소리로 합성하는 것은 향후 과제로 둔다.

기대 효과

- 직접 효과: 부모가 곁에 없는 순간에도 자녀가 부모의 말로 위로받는다. 자녀가 따로 찾지 않아도 평범한 한마디에 반응한다.
- 응용 분야: 애도 과정과 분리불안을 겪는 아동의 정서 지지, 치매 환자 가족 지원, 호스피스·완화의료의 생애 유산 기록, 장기 해외 근무 부모와 자녀의 연결
- 장기 목표: 심리상담 기관, 병원, 사회복지 서비스와 연계한 공공 서비스

대상 사용자(페르소나): 부모 한지은(41 세), 자녀 김민준(10 세, 초등 3 학년). 요구사항을 구체화하기 위해 설정한 가상 인물이며, 시연 데이터의 자녀 이름 "민준"도 여기서 가져왔다.

1.3 기능적 요구사항

구현 범위

저장된 부모 메시지가 자녀의 말에 맞춰 재생되는 한 사이클을 "구현"으로, 운영 환경에 필요한 나머지를 "설계"로 구분한다. 반드시 보여야 할 것은 시연 문장 4 개로 도는 한 사이클이다. LLM 이나 AWS 배포가 늦어지면 키워드·임베딩만으로 재생하거나 로컬 서버로 시연하는 대안을 미리 정해 두었다.

구분	구현	설계만
모바일 앱	Flutter 앱 하나에 부모 화면(메시지 등록·관리)과 자녀 화면(말하기, 진행 표시, 온기기록), 28 장	로그인, 계정 연결, 스피커 연결(페어링), 프로필 사진 변경, 보호자 등록
디바이스	음성 인식, 안전 확인, 키워드 규칙, 임베딩 판단, LLM 확인·공감 생성, 공감 규칙 검사, 부모 메시지 재생, 로컬 저장, 서버 동기화	위기 표현 감지 시 보호자 알림, 모델 무선 업데이트, 부모 목소리 합성
서버	FastAPI 서버 1 대: 메시지 저장, 디바이스 전달(API 14 개). 로컬(노트북, MySQL)에서 실행하고 같은 공유기로 연결한다. AWS(EC2, RDS, S3) 배포는 시간이 남으면 한다	IoT Core(MQTT)로 새 메시지 즉시 전달, 여러 가정 지원, 로그인, HTTPS

핵심 기능 목록

ID	구분	기능	설명	화면	범위
FR-P01	부모 앱	사용자 선택	시작 화면에서 부모/자녀를 선택한다	1-1, 1-2	구현
FR-P02	부모 앱	자녀 정보 설정	자녀 이름과 부모 호칭(엄마/아빠)을 입력한다	2-1	구현
FR-P03	부모 앱	첫 메시지 온보딩	추천 상황 하나를 골라 첫 메시지를 남기는 과정을 안내한다	2-2 ~ 2-7	구현
FR-P04	부모 앱	홈	추천 10 개 채우기 진행률과 4 개 분류 카드를 보여준다	3-1	구현
FR-P05	부모 앱	메시지 목록	저장된 메시지를 분류별로 보여주고 디바이스 전달 여부를 표시한다	3-2, 3-3	구현
FR-P06	부모 앱	메시지 남기기	상황 하나에 메시지 하나를 음성 녹음 또는 텍스트(200 자 이내)로 남긴다	2-4 ~ 2-6, 3-4, 3-5	구현
FR-P07	부모 앱	메시지 수정·삭제	저장한 메시지를 수정하거나 삭제한다	6-2	구현
FR-P08	부모 앱	설정	가족 정보와 온기 스피커 연결 상태를 확인한다	3-6	구현
FR-P09	부모 앱	보호자 등록	부모를 대신해 연락받을 보호자를 등록한다	-	설계
FR-P10	공통	로그인	계정으로 가족과 디바이스를 연결한다	1-1, 3-6	설계
FR-C01	자녀 앱	자녀 온보딩	말을 거는 방법과 다시 듣는 방법을 안내하고 남겨진 메시지 개수를 알려준다	4-1 ~ 4-4	구현
FR-C02	자녀 앱	홈	"온기에게 말하기" 버튼과 최근에 들은 메시지를 보여준다	5-1	구현
FR-C03	자녀 앱	말하기 진행 표시	듣는 중과 생각하는 중 상태를 보여주고, 받아쓴 글은 말이 끝난 뒤에 표시한다	5-2, 5-3	구현
FR-C04	자녀 앱	공감 한마디 표시	온기의 공감 한마디를 글로 함께 보여준다	5-4	구현
FR-C05	자녀 앱	부모 메시지 표시	재생 중인 부모 메시지를 보여준다. 텍스트는 글을, 녹음은 파형을 표시한다	5-5	구현
FR-C06	자녀 앱	맞는 메시지 없음	맞는 상황이나 메시지가 없을 때 받아주는 말을 보여준다	5-6	구현
FR-C07	자녀 앱	온기기록	이미 들은 메시지만 모아 보여준다	5-7	구현

ID	구분	기능	설명	화면	범위
FR-C08	자녀 앱	연결 상태 안내	스피커(디바이스)가 연결되지 않았을 때 안내한다	6-1	구현
FR-D01	디바이스	음성 구간 감지	마이크 입력에서 말이 끝난 것을 찾는다(silero VAD, 말 뒤 0.8 초 조용하면 끝)	-	구현
FR-D02	디바이스	받아쓰기	아이의 발화를 글자로 바꾼다(SenseVoice)	-	구현
FR-D03	디바이스	안전 확인	위기 표현을 규칙으로 먼저 검사해 정해진 안전 응답으로 연결한다	-	구현
FR-D04	디바이스	상황 판단	키워드로 상황을 정하고, 못 정하면 임베딩으로 정한다. 막는 말이 있으면 그 상황을 버린다	-	구현
FR-D05	디바이스	LLM 확인·공감 생성	LLM 이 상황·기분과 공감 한마디를 만들거나 일반 대화로 판정한다. 판단 규칙 A 를 적용한다	-	구현
FR-D06	디바이스	공감 한마디	생성 공감을 규칙으로 검사해 온기 목소리로 들려주고, 어기면 대체 공감을 쓴다	-	구현
FR-D07	디바이스	부모 메시지 재생	공감 → 연결 문장 → 부모 메시지를 한 파일로 재생한다. 녹음(m4a)은 풀어서, 텍스트는 TTS 로 미리 만들어 둔다	-	구현
FR-D08	디바이스	해당 없음 응답	상황을 정하지 못했거나 일반 대화면 받아주는 말을 들려준다	-	구현
FR-D09	디바이스	오프라인 동작	인터넷 없이 FR-D01 ~ FR-D08 을 수행한다	-	구현
FR-D10	디바이스	메시지 받기	30 초마다 서버에서 새로 생기거나 바뀌거나 지워진 메시지를 받아 반영한다	-	구현
FR-D11	디바이스	재생 기록	재생한 메시지와 시각을 기록한다. 발화 원문은 서버로 보내지 않는다	-	구현
FR-D12	디바이스	보호자 알림	위기 표현이 감지되면 등록된 보호자에게 알린다	-	설계
FR-S01	서버	메시지 저장	부모 메시지와 녹음 파일을 저장한다	-	구현
FR-S02	서버	디바이스 전달	디바이스가 요청하면 새 메시지와 재생 기록을 주고받는다	-	구현
FR-S03	서버	운영 환경	IoT Core(MQTT)로 새 메시지를 즉시 알리고 여러 가정을 지원한다	-	설계
FR-E01	부모 앱	자녀 이름 공유	부모가 정한 자녀 이름을 서버에 저장하고, 스피커와 자녀 앱이 같은 이름으로 부른다("서아한테 남긴 말이 있어")	2-1, 3-6	구현
FR-E02	부모 앱	스피커 연결 상태	부모 설정에 스피커의 마지막 연결 시각과 연결 여부(90 초 기준)를 보여 준다	3-6	구현
FR-E03	부모 앱	메시지 작성 유도	아이가 메시지 없는 상황을 말하면 상황 번호와 시각만 서버에 남기고, 부모 홈에 "이런 순간에 메시지를 찾았어요" 카드를 띄운다	3-1	구현
FR-E04	부모 앱	기록 지우기	부모가 재생 기록과 필요 기록을 지운다. 메시지는 그대로 둔다	3-6	구현

ID	구분	기능	설명	화면	범위
FR-E05	자녀 앱	재생 중 멈춤 확인	부모 메시지 재생 중에는 닫기(X)를 숨기고, "그만 들을래"와 뒤로 가기는 한 번 더 묻는다. 끝까지 들어야 온기기록에 남기 때문이다	5-5	구현

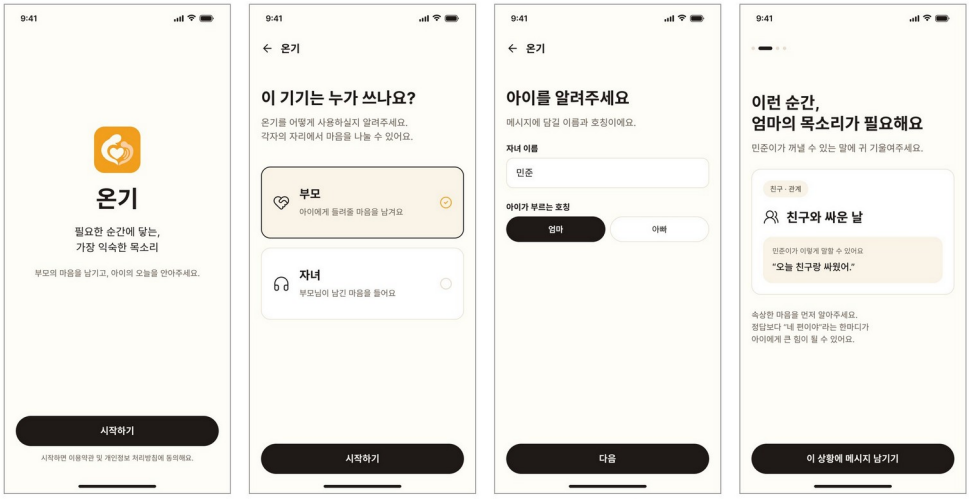
상황 구성: 부모가 고르는 상황과 AI 가 찾는 상황은 같은 12 개이며 분류마다 3 개씩이다(부록 A). 추천 10 개는 남들과 비교될 때(6)와 생일(12)을 뺀 10 개다. LLM 은 이 중 시연 상황 4 개(3, 4, 10, 11)를 학습했고, 나머지 8 개는 키워드와 임베딩으로 찾는다.

핵심 시나리오: 자녀가 "오늘 친구랑 싸웠어"라고 말한 경우다.

단계	처리 위치	내용
0. 사전	부모 앱 → 서버 → 디바이스	엄마가 "친구와 싸운 날(4)"에 메시지를 남기고, 디바이스가 30 초 안에 받아 저장하고 재생할 소리를 미리 만들어 둔다
1. 말 걸기	자녀 앱 → 디바이스	"온기에게 말하기"를 누르면 디바이스가 듣기 시작하고, 말이 끝나면 멈춘다
2. 받아쓰기	디바이스(SenseVoice)	"오늘 친구랑 싸웠어"로 받아쓴다
3. 안전 확인	디바이스	위기 표현이 없으므로 통과한다
4. 상황 판단	디바이스	키워드 "싸웠"이 상황 4 에 걸리고 막는 말이 없으므로 4 로 정한다
5. LLM 확인	디바이스(Genie, NPU)	`<maum_0>(situation=4, feeling=1)<maum_sep>`친구랑 싸워서 속상했구나`<maum_end>`를 낸다. 판단과 같은 상황이다
6. 공감 한마디	디바이스	생성 공감이 규칙을 통과하므로 "친구랑 싸워서 속상했구나"를 온기 목소리로 들려준다
7. 메시지 재생	디바이스	"엄마가 민준이한테 남긴 말이 있어. 들려줄게" 뒤에 엄마 메시지를 재생한다
8. 기록	디바이스 → 서버	재생한 메시지와 시각만 기록한다(API-14)

시연용 고정 데이터는 자녀 "민준"과 메시지 4 개(상황 3 엄마 글, 상황 4 엄마 녹음, 상황 10 엄마 녹음, 상황 11 아빠 글)다. 시연 문장은 "나 오늘 시험 망쳤어"(3), "오늘 친구랑 싸웠어"(4), "엄마 보고 싶어"(10), "나 오늘 상 받았어"(11), "배고파"(상황 없음, 받아주는 말)다. 부모 메시지는 "미리 남긴 말"이라는 성격에 맞춰, 오늘 일을 아는 것처럼 말하지 않고 부모의 에피소드를 하나씩 넣었다. 시연용 최종 문구는 팀원이 11 월 14 일까지 다시 쓴다.

화면 구성: 시작, 부모 온보딩, 부모 메인, 자녀 온보딩, 자녀 메인, 추가 상태의 6 개 구역 28 장으로 설계를 마쳤다(Figma). 시안은 24 개 상황일 때 만들어 3-1·3-3 의 분류 이름과 상황 수가 다르며, 앱 구현에서 12 개 상황과 분류 이름(학교·공부, 친구, 마음, 가족·특별한 날)으로 맞춘다.



1-1 시작 | 1-2 사용자 선택 | 2-1 자녀 정보 설정 | 2-3 추천 상황

2-4 녹음 전 | 2-5 녹음 중 | 2-6 녹음 후 | 2-7 첫 메시지 완료

<그림 1-1> 시작과 부모 온보딩: 사용자를 고르고 첫 메시지를 녹음해 남긴다



3-1 홈



3-2 메시지(전체)



3-3 메시지(분류 선택)



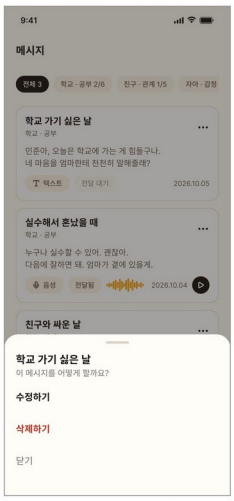
3-4 메시지 남기기(텍스트)



3-5 저장 완료



3-6 설정



6-2 수정·삭제 메뉴

<그림 1-2> 부모 메인: 추천 10 개 진행률, 메시지 목록과 전달 상태, 메시지 작성



4-1 환영



4-2 말하기 안내



4-3 다시 듣기 안내



4-4 준비 완료

<그림 1-3> 자녀 온보딩: 말을 거는 방법과 다시 듣는 방법을 안내한다



5-1 홈 | 5-2 듣는 중 | 5-3 생각하는 중 | 5-4 온기의 공감 한마디

5-5 부모 메시지 재생 | 5-6 맞는 메시지 없음 | 5-7 온기기록 | 6-1 스피커 연결 안 됨

<그림 1-4> 자녀 메인: 말 걸기부터 공감 한마디, 부모 메시지 재생, 온기기록까지

1.4 비기능적 요구사항

구분	요구사항	목표	측정 결과
성능: 응답 속도	자녀의 말이 끝난 뒤 첫 소리(공감 한마디)까지의 시간	3 초 이내	말 끝 확인 0.8 초 + 처리 0.82~1.02 초(실제 마이크·예제 LLM, 11/9) = 약 1.6~1.8 초. 양자화 모델로 다시 측정
성능: 상황 판단(LLM)	LLM 이 고른 상황(일반 대화 포함)이 정답과 같은 비율	90% 이상	95.6%(175/183, 병합 fp16, Colab). 보드 양자화 모델은 수령 후
성능: 잘못 재생	다른 상황이나 일반 대화에 부모 메시지가 재생되는 비율	5% 이하	테스트 결과표 일반·남 이야기 6 문장 중 0
성능: 답 형식	LLM 답이 형식을 지키는 비율	100%에 가까움	183/183(형식 깨짐 0)
성능: LLM 응답	LLM 한 번의 시간	1 초 이내	예제 모델 0.65~1.2 초. 학습 모델은 수령 후
성능: 동시 처리	디바이스 1 대가 한 번에 한 사람의 발화를 처리한다	1 건	충족

구분	요구사항	목표	측정 결과
자원	추론 중 CPU, 메모리 사용량	보드 실측 후 기준 설정	양자화 모델 수령 후 측정
신뢰성	해당 없음, 메시지 없음, 연결 끊김, LLM 실패에서 멈추지 않고 정해진 응답을 한다	오류 시나리오 전부 통과	시연 점검 9/9(규칙 A, 메시지 없음, 막는 말, 위기 표현 포함)
사용성	자녀는 버튼 한 번과 말 한마디로 메시지를 듣는다	-	앱 연결 시연 통과(11/9), 시연 영상 부모용 66 초·자녀용 57 초(11/12)
보안성	아이의 음성과 발화 원문은 서버로 보내지 않는다	전송 항목에 발화 원문 0 건	충족(API-14 는 메시지 ID 와 시각만)
확장성	상황 정의와 키워드는 파일 하나(situations.json)로 묶어 데이터만 고치면 반영되게 한다	-	충족
배포·업데이트	부모 메시지는 온라인일 때 자동으로 받아온다	30 초 안에 반영	충족(수정·삭제 포함, 실제 서버로 확인)

잘못 재생 비율을 가장 중요한 지표로 둔다. 엉뚱한 순간에 부모의 말이 나오는 것이 이 서비스에서 가장 나쁜 실패이기 때문이다.

응답 속도를 줄이는 방법

1. 모델(STT·TTS·임베딩·LLM)은 앱이 켜질 때 미리 올리고, STT 는 한 번 미리 돌려 둔다.
2. 부모 메시지와 연결 문장의 소리는 받을 때·앱을 켤 때 미리 만들어 둔다.
3. 말이 끝나면 0.8 초 안에 녹음을 멈춘다(VAD).
4. LLM 은 한 줄(번호 + 25 자 이내 문장)만 출력하고 끝 토큰에서 멈춘다.

오류 발생 시 처리 방안

상황	처리
상황을 정하지 못함 또는 LLM 이 일반 대화로 판정(키워드 없음)	받아주는 말을 들려준다(화면 5-6)
찾은 상황에 부모 메시지가 없음	생성 공감만 들려준다. 생성 공감이 규칙을 어기면 받아주는 말
키워드가 잡았는데 LLM 이 일반·다른 상황·형식 깨짐(판단 규칙 A)	키워드의 상황대로 대체 공감 + 부모 메시지
임베딩으로만 잡았고 LLM 과 다름	부모 메시지 없이 공감 한마디만
LLM 이 없거나 시간 초과(8 초)	키워드·임베딩으로 정한 상황대로 대체 공감 + 부모 메시지
받아쓴 글이 비어 있음	"잘 못 들었어. 한 번 더 말해 줘"를 소리로 안내한다
위기 표현 감지	부모 메시지 대신 안전 응답과 청소년 상담전화(1388) 안내를 들려준다
서버 연결 끊김	저장된 메시지로 계속 동작하고, 재생 기록은 모아 두었다가 나중에 올린다
앱과 디바이스의 연결 끊김	자녀 앱 홈에 연결 안 됨 상태를 표시한다(화면 6-1)

안전 및 윤리 고려

- 온기는 치료나 상담을 대신하지 않는다. 아이의 일상에서 정서적 지지를 돕는 보조 도구로 한정한다.
- AI 가 부모를 흉내 내지 않는다. 부모 메시지는 부모가 쓴 말만 재생하고, AI 가 만드는 공감 한마디는 온기의 목소리로 구분한다. 공감 문장에는 부모 호칭, 조언, 약속을 넣지 않는다.
- 위기 표현은 규칙으로 먼저 걸러 안전 응답과 상담전화 안내로 연결한다.
- 핵심 대상이 부모와 떨어진 아이이므로, 부모를 대신해 연락받을 보호자를 등록하는 기능을 설계에 둔다.

1.5 환경 요구사항

항목	내용
보드	Coretronic COREKOM R6490Q(Qualcomm QCS6490 계열, SoC SM7325), SUDA 음성 AI 키트
프로세서	Kryo 670 CPU 8 코어, Adreno 643 GPU, Hexagon NPU(HTP V68)
메모리 / 저장	약 8GB / 94GB
OS	Android 13(userdebug, root 가능). 디바이스 프로그램은 Kotlin 안드로이드 앱
주변 장치	핀마이크(ABKO HM10), 미니 스피커(ABKO SP1), USB 사운드카드(GeneralPlus, 재생 스테레오·녹음 모노, 44.1·48kHz), 공유기(ipTIME N604SR), USB-C 허브
모바일 앱	Android 스마트폰(시연 폰)
서버	노트북 1 대(Python 3.13, FastAPI, MySQL 8.0.16 이상), 같은 공유기
LLM 학습	Google Colab(T4 GPU), 양자화·변환은 (주)마음 AI

1.6 제약사항

구분	내용
일정	최종평가 및 시연 11 월 26 일(13 주차), SW 페스티벌 11 월 27 일. 구현 기간은 10 월 22 일부터 5 주
인력	4 명, 파트별 1 명
기술	LLM 은 Genie 가 지원하는 모델(Llama-3.2-1B), 문맥 128 토큰 안에서 입력과 출력을 끝내야 한다. 양자화는 마음 AI 가 맡는다
자원	디바이스는 배정받은 1 대로 개발과 시연을 모두 진행한다
범위	로그인, 페어링, 보호자 등록, IoT Core, 부모 목소리 합성은 설계까지만 한다. 가족은 하나(family_id = 1)로 고정한다
시연	저장된 부모 메시지가 재생되는 한 사이클을 시연 문장 4 개로 보이고 영상으로 기록한다

한계와 향후 과제

한계	대응 및 향후 과제
LLM 은 4 개 상황만 학습했다	나머지 8 개는 키워드·임베딩으로 찾는다. 학습 데이터를 늘려 12 개로 넓히는 것을 향후 과제로 둔다
데이터가 모두 가상으로 만든 문장이다	평가 문장을 학습과 분리해 작성하고, 향후 실제 사용 발화로 보완한다
아동 음성으로 검증하지 못한다	시연과 테스트는 팀원 음성으로 진행한다

한계	대응 및 향후 과제
부모 메시지가 부모의 실제 목소리가 아니다 (시연은 온기 목소리)	부모 녹음 재생은 구현돼 있다. 부모 목소리 TTS 학습은 향후 과제로 둔다
상황 하나에 메시지가 하나다	상황당 여러 메시지와 나이대별 메시지는 향후 과제로 둔다
아이가 말을 걸어야만 동작한다	생일처럼 날짜가 정해진 순간에 온기가 먼저 말을 거는 기능을 향후 과제로 둔다
심리적 영향에 대한 전문가 검토가 없다	아동 상담 분야의 자문과 실제 사용자 검증을 향후 과제로 둔다

활용 분야

온기의 구조(보호자가 미리 남기고, 기기가 상황을 알아듣고, 기기 안에서 들려준다)는 보호자와 아이가 떨어져 있는 여러 상황에 그대로 쓸 수 있다. 아이의 말이 기기 밖으로 나가지 않으므로 병원이나 시설처럼 개인정보에 민감한 곳에도 맞다. 아래는 같은 구조로 쓸 수 있는 곳이다.

대상	상황	온기가 하는 일
장기 출장·해외 근무·교대 근무 부모	아이가 잠들기 전이나 힘든 날 부모가 곁에 없다	상황별로 남긴 부모 목소리를 그 순간에 들려준다
오래 입원한 아이	검사나 치료 전에 무섭고, 보호자가 늘 곁에 있기 어렵다	무서울 때, 잠이 오지 않을 때 가족의 목소리를 들려준다
부모를 잃었거나 부모가 투병 중인 가정	부모가 목소리를 남길 수 있는 시간이 정해져 있다	생일, 졸업처럼 아이가 자라며 맞을 순간에 남긴 목소리를 전한다
조부모 돌봄·위탁가정·보육시설	양육자가 바뀌거나 한 사람이 여러 아이를 돌본다	원래 보호자나 담당 선생님의 목소리로 일상의 순간을 이어 준다
혼자 지내는 노부모	자녀와 떨어져 지내며 외로운 시간이 길다	자녀가 남긴 목소리를 외롭거나 힘든 순간에 들려준다(방향만 바뀐 같은 구조)

2. 아키텍처 설계서

온기 시스템의 전체 구조와 구성 요소를 정의하고, 하드웨어와 소프트웨어가 어떻게 맞물려 동작하는지 밝힌다.

2.1 문서 개요

본 문서는 모바일 앱, 서버, 온기 디바이스 세 계층의 구조와 그 사이의 데이터 흐름을 다룬다. 모듈 사이의 호출 규격은 4 장, 모델에 대한 상세는 3 장에서 다룬다.

2.2 시스템 개요

설계 목표 및 원칙

1. 디바이스가 원본 보관소다. 부모 메시지는 디바이스에 저장되고, 서버가 없어져도 이미 받은 메시지는 재생된다.
2. 알아듣고 판단하고 들려주는 과정은 전부 디바이스 안에서 처리한다.
3. 상황은 키워드가 우선이다. 키워드와 임베딩이 상황을 정하고, LLM 이 한 번 더 확인하며 공감 한마디를 만든다. LLM 이 틀려도 키워드가 잡은 상황은 재생된다(판단 규칙 A).
4. 아이의 말은 서버로 보내지 않는다. 서버로 가는 것은 재생한 메시지와 시각뿐이다.
5. 제한된 자원에 맞춘다. LLM 은 NPU 에, STT·TTS·임베딩은 CPU 에 두고, 모두 디바이스 앱이 켜질 때 미리 올려 둔다.

디바이스를 따로 두는 이유

- 집에 놓인 유산함이다. 스마트폰은 몇 년마다 바뀌지만 디바이스는 부모의 메시지를 담은 채 한자리에 남는다.
- NPU 에서 LLM 을 돌린다. 판단과 공감에 쓰는 LLM 을 기기 안에서 실행하려면 전용 가속기가 있는 보드가 필요하다.
- 앱은 창구 역할만 한다. 앱이 없어도 디바이스에 저장된 메시지와 판단 기능은 그대로 유지된다.

계층	역할
모바일 앱	부모는 메시지를 남기고 관리한다. 자녀는 말하기를 시작하고 진행 상태와 들은 메시지를 본다
서버	부모가 남긴 메시지와 녹음 파일을 저장하고, 디바이스가 요청하면 전달한다
온기 디바이스	마이크로 들은 말을 인식하고, 상황을 판단하고, 공감 한마디와 부모 메시지를 스피커로 들려준다

2.3 하드웨어 아키텍처

구성 요소	역할
QCS6490 보드	음성 인식, 상황 판단, 음성 출력을 수행하는 본체
핀마이크 + USB 사운드카드	자녀의 발화 입력(48kHz 모노 녹음 후 16kHz 로 줄임)
미니 스피커	공감 한마디와 부모 메시지 출력(48kHz 스테레오)
공유기	앱, 디바이스, 서버 노트북을 같은 네트워크로 연결(ipTIME N604SR)

사운드카드는 보드가 부팅을 마친 뒤 보드 USB-A 에 직접 꽂는다. 꽂은 채 부팅하면 카드 번호가 바뀐다. 이 보드에서는 Android 앱 경로(AudioTrack·AudioRecord)로 USB 사운드카드를 쓸 수 없어(USB 오디오 오프로드), root 권한의 오디오 도우미가 tinyplay·tinycap 으로 하드웨어를 직접 쓴다. 앱은 요청 파일만 쓰고 도우미가 재생·녹음 결과를 남긴다.

연산	배치	이유
VAD, STT	CPU	연산량이 작다(STT 0.2 초 안팎)
임베딩(MiniLM int8)	CPU	모델이 작고 기준 문장 1,473 개와의 유사도 계산도 가볍다
LLM(Llama-3.2-1B)	NPU(HTP)	가장 무거운 연산이라 Genie 를 통해 NPU 에서 실행한다
TTS(VITS)	CPU	부모 메시지·연결 문장은 미리 만들고, 공감 한마디만 그때 만든다(0.3 초 안팎)

2.4 소프트웨어 아키텍처

레이어	구성	위치
앱 레이어	부모 화면, 자녀 화면과 상태 관리	모바일(Flutter)
서비스 레이어	메시지 API 와 저장 / 디바이스의 판단·재생 흐름	서버(FastAPI), 디바이스 앱(Kotlin)
모델 레이어	STT, TTS, 임베딩, LLM	디바이스 앱 안(Genie, sherpa-onnx, ONNX Runtime)
데이터 레이어	MySQL, 녹음 파일 / SQLite	서버 / 디바이스

디바이스 앱 모듈 구성(패키지 `com.aiq.ongi.device`)

모듈	역할
flow	전체 대화 흐름과 상태 전환. 듣기 → 받아쓰기 → 위기 → 판단 → LLM → 재생 규칙 → 소리 준비 → 신호·스피커
safety	위기 표현 규칙 검사
judge	키워드 판단(막는 말 포함), 임베딩 판단(기준 문장 비교), situations.json 읽기
ai_client	모델 호출을 한곳에 모음. Genie(LLM), SenseVoice·VITS(sherpa-onnx), MiniLM(ONNX Runtime), 오디오 변환(m4a 풀기, 48kHz 스테레오)
respond	LLM 답 해석, 판단 규칙 A, 공감 규칙 검사, 대체 공감
audio	root 오디오 도우미에 재생·녹음 요청, 마이크 말 끝 판단(silero VAD)
store	부모 메시지, 재생 기록, 동기화 상태(SQLite)
sync	30 초마다 서버에서 변경분을 받아 저장하고 재생 기록을 올림. 서버 주소가 바뀌면 처음부터 다시 받음
bridge	자녀 앱의 시작 요청을 받고 진행 상태를 알림(WebSocket 8765)
오디오 도우미	보드에서 root 로 도는 셸 스크립트. tinyplay·tinycap 으로 USB 사운드카드를 직접 사용



<그림 2-1> 온기 동작 흐름: 아이 말은 보드 안에서 판단하고, 서버와는 부모 메시지와 재생 기록만 오간다

아이의 말은 보드 안에서 받아쓰기부터 판단, 공감 한마디까지 끝나고, 서버와는 부모 메시지(30 초마다 동기화)와 재생 기록만 오간다.

재생 규칙

경우	처리
판단한 상황 = LLM 의 상황	생성 공감(규칙 통과 시) + 부모 메시지
키워드가 잡았는데 LLM 이 일반·다른 상황·형식 깨짐(판단 규칙 A)	키워드의 상황대로 대체 공감 + 부모 메시지
임베딩으로만 잡았고 LLM 과 다름	공감 한마디만
상황 없음 + LLM 일반 대화	받아주는 말
LLM 없음·시간 초과	판단한 상황대로 대체 공감 + 부모 메시지
그 상황에 부모 메시지 없음	생성 공감만(규칙 위반이면 받아주는 말)
여러 상황의 키워드가 걸림	생일(12) > 사건(2~6, 11) > 그리움(10) > 반응(1, 7~9)

2.5 데이터 아키텍처

저장소	위치	보관 내용	범위
SQLite	디바이스	부모 메시지(글, 녹음 파일 경로), 재생 기록(서버 전송 여부), 동기화 상태(since, 서버 주소, 서버 시각 보정값)	구현
모델·데이터 파일	디바이스	situations.json(상황, 키워드, 막는 말, 위기 표현), 기준 문장 벡터(1,473 개), 모델 파일	구현
설정 파일	디바이스	config.json(서버 주소, 아이 이름, LLM 설정). 빌드 없이 바꾼다	구현
MySQL	서버	가족, 상황 12 개, 부모 메시지(전달 여부, 삭제 표시), 재생 기록	구현
녹음 파일	서버	부모 녹음(m4a). 서버가 만든 임의 이름으로 저장	구현
AWS RDS·S3, IoT Core	서버	운영 환경의 DB·파일 저장과 실시간 전달	설계

처리 방식

- 음성 입력: 마이크 녹음을 0.1 초마다 읽어 말 끝을 판단하고, 발화 한 건 단위로 처리한다.
- 메시지 받기: 30 초마다 마지막으로 받은 서버 시각 이후의 변경분을 받는다. 삭제된 메시지는 녹음 파일까지 지운다. 서버와 한 번이라도 동기화한 뒤에는 시연용 기본 메시지를 다시 채우지 않는다.
- 미리 만들기: 부모 메시지(녹음은 풀고, 글은 TTS)와 연결 문장의 소리를 받을 때·앱을 켤 때 만들어 둔다.
- 재생 기록: 디바이스에 먼저 쌓고, 서버에 닿으면 보낸다. 보드는 시계가 따로 돌지 않아 서버 시각 보정값으로 재생 시각을 적는다.

데이터	디바이스 밖으로 나가는가	설명
아이의 음성	나가지 않음	받아쓰기가 끝나면 보관하지 않는다
아이의 발화 글	서버로 나가지 않음	화면 표시를 위해 같은 공유기 안의 자녀 앱에만 전달한다
재생 기록	메시지 ID 와 시각만 전송	발화 원문은 포함하지 않는다
부모 메시지	서버에서 디바이스로 전달	부모가 직접 등록한 내용이다

2.6 주요 기술 스택

영역	기술	범위
모바일 앱	Flutter(Dart), WebSocket, 녹음(m4a)	구현
서버	Python 3.13, FastAPI, PyMySQL, MySQL 8.0.16 이상	구현
디바이스 앱	Kotlin(Android 13), SQLite, Java-WebSocket	구현
통신	HTTP/REST(앱 ↔ 서버, 디바이스 ↔ 서버), WebSocket(자녀 앱 ↔ 디바이스)	구현
음성	silero VAD, SenseVoice STT, VITS TTS(sherpa-onnx 1.12.7, SUDA 앱과 같은 모델)	구현
오디오	tinyalsa(tinyplay·tinycap), root 도우미 스크립트, MediaCodec(m4a 풀기)	구현
임베딩	paraphrase-multilingual-MiniLM-L12-v2(384 차원), ONNX int8, onnxruntime-android 1.17.1	구현
LLM	Llama-3.2-1B-Instruct, Qualcomm Genie(QNN), SUDA 앱 라이브러리	구현
학습	PyTorch, Hugging Face Transformers·PEFT(LoRA), Google Colab T4	구현
배포	로컬 노트북 서버 + 공유기. AWS EC2·RDS·S3 는 시간이 남으면	구현 / 선택
운영 환경	AWS IoT Core(MQTT), 여러 가정, 로그인, HTTPS	설계
협업	GitHub 저장소 1 개(app/, server/, device/, ai/, docs/), Figma	-

3. 모델 설계 및 최적화 문서

QCS6490 환경에 맞는 AI 모델을 고르고, 데이터 구성과 학습, 경량화 전략을 정리해 디바이스에서 목표 성능을 확보하는 방법을 기록한다.

3.1 문서 개요

본 문서는 온기 디바이스에서 동작하는 모델 5 종과 그 데이터, 학습, 평가, 경량화 방법을 다룬다. 현재 학습 데이터 v3.7 을 확정하고 LLM 학습·병합·채점을 마쳐 양자화를 맡긴 상태이며, STT·TTS·임베딩은 보드에서 동작한다.

3.2 모델 선정 및 개요

역할	모델	구조	선정 이유
음성 구간 감지	silero VAD	경량 신경망	SUDA 앱과 같은 sherpa-onnx 스택에 포함
받아쓰기	SenseVoice	음성 인식 모델	한국어 지원, CPU 에서 0.2 초 안팎. SUDA 앱 모델을 그대로 쓴다
상황 판단(키워드 다음)	paraphrase-multilingual-MiniLM-L12-v2	Transformer 인코더(384 차원)	한국어를 포함한 다국어 모델이고 int8 로 줄여 CPU 에서 돈다
상황·기분 확인, 공감 생성	Llama-3.2-1B-Instruct	Transformer 디코더(LLM)	Genie 가 지원하는 공개 모델 중 작고, 짧은 형식의 출력은 안정적으로 학습된다
음성 합성	VITS(한국어 여성 화자 1 명, 22,050Hz)	음성 합성 모델	SUDA 앱과 같은 모델. 다른 TTS 는 찾지 않기로 했다

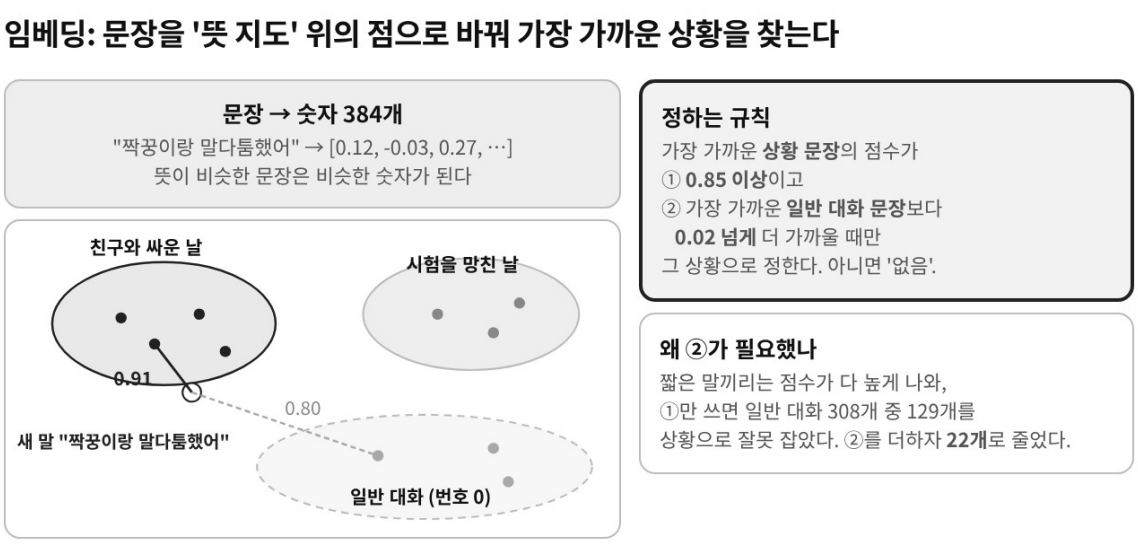
상황을 여러 방식으로 판단하는 이유

상황 판단은 잘못 재생을 막는 쪽을 우선한다. 키워드가 있는 말은 규칙으로 정하고, 막는 말로 남의 일과 다른 뜻을 걸러낸다. 키워드가 없는 말은 임베딩이 충분히 가까울 때만 정한다. LLM 은 문장 전체를 읽고 상황을 다시 고르며 공감 한마디를 만든다.

방식	장점	한계
키워드만	빠르고 판단 이유를 설명할 수 있다. 학습·평가 데이터에서 틀린 적이 없다	키워드가 없는 말을 놓친다
LLM 만	문장 전체의 뜻을 읽는다	1B 모델이 틀리면 막을 장치가 없고, 보드에서 돌지 않으면 판단이 멈춘다
키워드·임베딩 + LLM, 판단 규칙 A(채택)	키워드가 잡은 상황은 항상 재생되고, LLM 이 맞으면 아이 말에 맞는 공감이 나온다. LLM 없이도 동작한다	임베딩으로만 잡은 상황은 LLM 과 같아야 부모 메시지가 나온다

임베딩 판단은 이렇게 준비했다

키워드에 걸리지 않는 말(예: "짝꿍이랑 말다툼했어")은 임베딩으로 찾는다. 임베딩 모델은 문장을 숫자 384개로 바꾸는데, 뜻이 비슷한 문장은 비슷한 숫자가 된다. 이 숫자를 지도 위의 좌표처럼 생각하면, 새로 들은 말과 가장 가까이 있는 예시 문장의 상황이 곧 아이가 말한 상황이다.



그림은 이해를 돕기 위한 예시다(점수 값은 설명용). 실제 판단은 384차원에서 코사인 유사도로 계산한다.

<그림 3-1> 임베딩 판단: 새 말과 가장 가까운 예시 문장을 찾되, 일반 대화보다 확실히 가까울 때만 상황으로 정한다

임베딩 모델은 따로 학습하지 않았다. 공개된 다국어 모델을 그대로 쓰고, 어떤 예시 문장을 넣을지와 얼마나 가까워야 인정할지(기준값)로 성능을 맞췄다. 학습 없이도 예시 문장만 바꾸면 고칠 수 있고, LLM 이 없을 때도 시연이 돌아가는 바탕이 된다. 준비는 다음 순서로 했다.

단계	한 일	이유와 결과
1. 모델 고르기	paraphrase-multilingual-MiniLM-L12-v2(384 차원)를 골랐다	한국어를 지원하는 문장 비교 모델 중 작고 빠르다
2. 가볍게 만들기	Colab 에서 ONNX 로 바꾸고 int8 로 줄였다. 평균 내기와 길이 맞추기(정규화)까지 모델 안에 넣었다	보드 CPU 에서 바로 돈다(파일 118MB). 보드에서는 두 벡터의 곱만 하면 유사도가 나온다
3. 토크나이저 직접 만들기	문장을 조각으로 나누는 토크나이저 (SentencePiece Unigram)를 디바이스 앱 안에 직접 구현했다	변환 도구(onnxruntime-extensions)가 새 형식을 변환하지 못했다. 학습·평가 문장 전부에서 원래 토크나이저와 결과가 같은지 Colab 에서 확인했다
4. 예시 문장 미리 넣기	학습 데이터 v3.5.2 의 1,473 문장을 미리 벡터로 바꿔 파일로 넣었다(상황 1~12, 일반·불명확은 0 번)	말할 때마다 계산하지 않고 비교만 한다 (2.2MB)
5. 기준값 정하기	가장 가까운 상황 문장이 0.85 이상이고, 가장 가까운 일반 문장보다 0.02 넘게 가까울 때만 정한다	점수 하나만 보면 일반 대화 308 개 중 129 개를 잘못 잡았고, 차이 조건을 더하자 22 개로 줄었다
6. 확인	평가 문장으로 채점하고, 같은 문장이 PC 와 보드에서 같은 결과가 나오는지 비교했다	평가 상황 문장 96.6% 정답, 다른 상황으로 착각 1, 일반을 잘못 잡음 1(11/2)

12 개 상황을 그대로 둔 기준 문장과 시연 4 개 상황만 남긴 기준 문장을 비교했을 때 평가 상황 문장에서 둘 다 120/120 이어서, 나중에 상황을 늘리기 쉬운 12 개 구성을 유지했다(3.3 절).

LLM 입출력 형식

구분	내용
입력	`<|im_start|>` + 마음 AI 템플릿 + 아이 말 + `Response:` . 템플릿은 "Below is the query from the users, please choose the correct function and generate the parameters to call the function. Query: {발화}"
출력(상황 말하기)	`<maum_0>(situation=4, feeling=1)<maum_sep>`친구랑 싸워서 속상했구나`<maum_end>`
출력(일반 대화)	`<maum_1>()<maum_end>`
새 토큰	`<maum_0>` 128256, `<maum_1>` 128257, `<maum_sep>` 128258, `<maum_end>` 128259(vocab 128260). 멈춤 토큰은 `<maum_end>`와 `<|eot_id|>`

입력 앞머리 `<|im_start|>`는 보드의 SUDA 앱과 같게 맞춘 것이며, 학습·평가·보드 추론에서 똑같이 쓴다. 보드에서는 설정 파일로 빌드 없이 바꿀 수 있다. 상황 후보나 상황 이름은 입력에 넣지 않는다.

공감 한마디 규칙(학습 정답 작성과 보드 검사에 같이 쓴다)

- 한 문장, 띄어쓰기를 뺀 25 자 이내, 반말
- 한글과 공백만, 물음표 금지
- 조언·지시("~해 봐"), 약속("~할게") 금지
- 부모 호칭(엄마·아빠·부모님)은 아이 말에 같은 호칭이 있을 때만 허용
- 같은 문장은 학습 데이터에 3 번까지

생성 공감이 규칙을 어기면 시연 상황마다 3 개씩 미리 쓴 대체 공감(총 12 개, 학습 정답과 다른 문장)을 번갈아 쓴다. 받아주는 말과 안전 응답은 LLM 이 만들지 않는 고정 문장이다.

기분 번호	기분	공감 한마디 예시
1	슬픔·속상함	시험을 망쳐서 속상했구나
5	외로움·그리움	엄마가 많이 보고 싶구나
6	기쁨·뿌듯함	오늘 상을 받은 날이구나
-	상황 없음·일반 대화(받아주는 말)	그런 날도 있지, 이야기해 줘서 고마워

3.3 데이터셋 구성

학습 데이터는 **v3.7 로 확정**했다(11/5). 설계·키워드·임베딩은 12 개 상황을 유지하고, LLM 학습과 시연은 4 개 상황(3, 4, 10, 11)에 집중했다. 나머지 상황의 말과 남의 일·놀이 같은 경계 밖 문장은 일반 대화(`<maum_1>`)로 학습한다.

항목	값
학습 데이터	1,061 문장 = 원본 849 + 띄어쓰기 변형 212(음성 인식처럼 "친 구랑", "시험망쳤어". 내용과 정답은 그대로)

항목	값
평가 데이터	183 문장 = 상황 120(4 개 상황 각 30) + 일반 대화 63. 학습과 겹침 0
기분	상황마다 고정(시험 1, 친구 1, 보고 싶음 5, 자랑 6)
형식	마음 AI 학습 형식(No, 기능명, LLM Output, Query) CSV · JSON
키워드	12 개 상황, situations.json. 학습 데이터 상황 문장 745/745, 평가 120/120 을 맞힌다
임베딩 기준 문장	1,473 문장(v3.5.2, 12 개 상황 + 없음). 임계값 0.85, 없음과의 차이 0.02

작성 원칙

- 상황 문장은 사건 단어와 마음이 드러나는 내 일만 넣는다. 장소 · 이유 · 특정 인물 같은 변수는 빼고 정한 말만 쓴다(11/4).
- 책 · 만화를 보다 운 말처럼 애매한 말과 두 상황이 섞인 말은 학습 · 평가에서 뺀다.
- 문장부호, 이모티콘, 영어, 숫자를 넣지 않고 한글과 공백만 쓴다. 실제 입력이 음성 인식 결과이기 때문이다.
- 평가 문장은 학습에 없는 표현으로 쓰고, 처음 보는 말투에도 맞히는지 보려고 "있잖아", "~단 말이야" 같은 말투를 학습보다 많이 넣었다. 학습 데이터는 표지 없는 평범한 말이 46%, 평가 데이터는 30%다.
- 학습 데이터는 스크립트로만 만들고 검사한다. 키워드를 더할 때는 학습 · 평가 데이터의 판단이 바뀌지 않는지 먼저 확인한다.

12 개 상황을 그대로 두고 4 개로도 임베딩 기준 문장을 만들어 비교했지만, 평가 상황 문장에서 둘 다 120/120 으로 차이가 없어 12 개(v3.5.2) 기준을 유지했다.

3.4 모델 학습 및 평가 전략

학습하는 모델은 Llama-3.2-1B-Instruct 하나다. 새 토큰 4 개를 추가하고 `<maum_end>`를 멈춤 토큰으로 두어 위의 출력 형식을 학습시켰다. MiniLM 은 학습하지 않고 기준 문장과 임계값으로 조정한다.

항목	값
학습 환경	Google Colab T4 GPU. 멘토 GPU 를 받지 못해 팀이 직접 학습했다
베이스 모델	unsloth/Llama-3.2-1B-Instruct(Meta 원본과 같은 가중치, 허가 없이 받을 수 있는 사본)
학습 방식	LoRA r=16, 모든 선형층 + 새 토큰 4 개의 임베딩
Epoch / Batch / Learning Rate	3 / 8 / 2e-4
제출물	LoRA 를 베이스와 병합한 fp16 모델(2.47GB) + 토크나이저 + 학습 · 평가 데이터. 11/5 마음 AI 공유 폴더에 제출
채점 기준	형식(정해진 모양인가), 상황(상황 번호가 같은가), 일반(일반 대화를 일반으로 봤는가), 공감 규칙 위반

학습 결과(평가 183 문장, 병합 fp16)

항목	결과	목표
전체 정답	175/183(95.6%)	90% 이상

항목	결과	목표
상황 문장	113/120(94.2%)	95%
일반 대화	62/63(98.4%)	-
답 형식 깨짐	0	0%에 가까움

LoRA 와 병합 모델의 결과는 같았다. 틀린 8 문장 중 7 개는 상황을 일반으로 본 경우("있잖아 아빠 그립단 말이야" 등)이며, 보드에서는 키워드와 판단 규칙 A 가 받아 부모 메시지가 재생된다. 1 개는 남의 일을 상황으로 본 경우("형이 친구랑 싸웠대")로, 보드에서는 막는 말이 걸러낸다. 공감에 아이가 부른 호칭이 앞에 붙는 경우가 3 개("엄마 친구랑 싸웠구나") 있었고, 보드에서 거를지는 양자화 모델 평가 뒤에 정한다.

보드 평가: 양자화 모델을 받으면 보드 평가 도구로 같은 평가 183 문장을 보드 LLM 에 넣어 같은 기준으로 채점하고, LLM 시간도 함께 잰다. Colab 결과와 크게 다르면 입력 앞머리를 바꿔 비교하고, 그래도 다르면 재학습을 검토한다.

3.5 모델 경량화 및 최적화 방안

대상	기법	내용
Llama-3.2-1B	양자화·변환	(주)마음 AI 가 병합 fp16 모델을 양자화해 QNN context binary 와 Genie 설정 파일로 만든다
Llama-3.2-1B	입출력 길이 제한	보드 Genie 의 문맥은 128 토큰이다. 고정 부분 약 29 토큰 + 출력 최대 30 토큰을 빼고, 아이 말은 60 자로 자른다
Llama-3.2-1B	멈춤 토큰	Genie 설정의 eos 에 `<maum_end>`(128259)를 넣어 공감 문장 끝에서 바로 멈추게 한다
MiniLM	int8 ONNX	토크나이저(SentencePiece Unigram)는 앱에서 직접 구현하고, 모델은 int8 ONNX 로 줄여 CPU 에서 실행한다
전체	미리 올리기·미리 만들기	앱이 켜질 때 모든 모델을 올리고, 부모 메시지 소리를 미리 만든다(약 9 초)

가지치기와 지식 증류는 적용하지 않는다. 1B 모델은 양자화만으로 디바이스에 올릴 수 있는 크기다.

항목	목표	현재
전체 응답 시간	3 초 이내	1.6~1.8 초(가짜 LLM)
LLM 시간	1 초 이내	예제 모델 0.65~1.2 초, 학습 모델은 수령 후
양자화 전후 정확도 차이	3%p 이내	수령 후 보드 평가 도구로 측정

3.6 추론 엔진 및 런타임 환경

모델	추론 엔진	실행 장치	모델 형식
silero VAD	sherpa-onnx 1.12.7	CPU	ONNX
SenseVoice	sherpa-onnx 1.12.7	CPU	ONNX(int8)
MiniLM	onnxruntime-android 1.17.1	CPU	ONNX(int8)

모델	추론 엔진	실행 장치	모델 형식
Llama-3.2-1B	Qualcomm Genie(QNN 2.28)	NPU(HTP)	QNN context binary
VITS	sherpa-onnx 1.12.7	CPU	ONNX

모든 엔진은 디바이스 앱 안에 라이브러리로 들어 있다. Genie 와 sherpa-onnx 라이브러리는 보드에 들어 있던 SUDA 앱에서 가져왔고, onnxruntime 버전은 sherpa-onnx 와 맞추려고 1.17.1 로 고정했다.

4. API 및 인터페이스 정의서

온기의 구성 요소가 서로 데이터를 주고받는 방법을 정의해, 앱, 서버, 디바이스를 맡은 팀원이 같은 규격으로 개발할 수 있게 한다.

4.1 문서 개요

본 문서는 앱과 서버, 디바이스와 서버, 자녀 앱과 디바이스 사이의 인터페이스와 디바이스 앱 안의 모델 호출을 다룬다. 공통 형식은 시각 `2026-11-02T20:15:00`(초까지, 시간대 표기 없이 한국 시각), 데이터 키는 영어 소문자와 밑줄, 문자 인코딩 UTF-8 이다. API 는 서버·DB 파트의 API 규격서 v0.6(server 브랜치 `server/API.md`), 신호는 디바이스 파트의 신호 규격서를 따른다. 바꾸려면 팀 단톡방에 [규격 변경]으로 알리고 영향받는 사람의 확인을 받는다.

4.2 인터페이스 개요

ID	구간	방식	용도
IF-1	모바일 앱 ↔ 서버	HTTP(REST), JSON	가족 정보, 메시지 등록·조회·수정·삭제, 녹음 업로드, 진행률, 재생 기록
IF-2	디바이스 ↔ 서버	HTTP(REST), JSON	변경분 받기, 녹음 내려받기, 전달 확인, 재생 기록 올리기
IF-3	자녀 앱 ↔ 디바이스	WebSocket, JSON	말하기 시작 요청, 진행 상태 알림
IF-4	디바이스 앱 안	함수 호출(라이브러리)	받아쓰기, 음성 합성, 임베딩, LLM

부모 앱은 IF-1 로 메시지를 서버에 등록하고, 디바이스는 IF-2 로 30 초마다 새 메시지를 받아온다. 자녀 앱은 같은 공유기 안에서 IF-3 으로 디바이스에 직접 연결한다. 앱은 모델을 직접 부르지 않는다.

4.3 API 목록 및 상세 정의

IF-1 모바일 앱 ↔ 서버

ID	메서드	주소	기능	응답·오류	화면
API-01	POST	/families	자녀 이름과 엄마·아빠 호칭을 등록한다. 호칭을 비우면 "엄마", "아빠"	201 / 가족이 이미 있으면 E-409	2-1
API-02	GET	/situations	상황 12 개를 조회한다	`{"items":[…]}`	2-3, 3-1, 3-3
API-03	GET	/messages?family_id=1	지워지지 않은 메시지를 상황 번호 순으로 조회한다(전달 여부 포함)	`{"items":[…]}` / E-404	3-2, 3-3
API-04	POST	/messages	상황 하나에 메시지를 등록한다	201 / E-400, E-404, E-409	2-6, 3-4

ID	메서드	주소	기능	응답·오류	화면
API-05	POST	/audio	녹음 파일(m4a, 최대 10MB)을 올린다. 메시지는 API-04 로 따로 등록	201 `{"audio_path","size_bytes","content_type"}` / E-400, E-413	2-6
API-06	PATCH	/messages/{id}	바꿀 칸만 보내 수정한다. 전달 여부를 되돌린다	200 / E-400, E-404	6-2
API-07	DELETE	/messages/{id}	삭제 표시(소프트 삭제)한다	200 / E-404	6-2
API-08	GET	/families/{id}/progress	추천 10 개 중 채운 개수를 조회한다	`{"completed","total":10}`	2-7, 3-1, 3-5
API-09	GET	/play-logs?family_id=1	자녀가 들은 메시지 기록을 최근 순으로 조회한다	`{"items":[…]}`	5-1, 5-7
API-10	GET	/messages/count?family_id=1	남겨진 메시지의 개수만 조회한다	`{"count"}`	4-1
API-15	GET	/families/{id}	가족 정보와 스피커 마지막 연결 시각, 연결 여부(90 초 안이면 true)를 조회한다(11/11)	`200 / E-404`	3-6
API-16	PATCH	/families/{id}	자녀 이름, 엄마·아빠 호칭 중 바꿀 칸만 고친다(11/11)	`200 / E-400, E-404`	2-1, 3-6
API-18	GET	/needs?family_id=1	최근 14 일 동안 아이가 찾았는데 지금도 메시지가 없는 상황을 조회한다(11/11)	`{"items":[…]}`	3-1
API-19	DELETE	/families/{id}/history	재생 기록과 필요 기록을 지운다. 메시지는 그대로 둔다(11/11)	`200`	3-6

API-04 메시지 등록 요청

이름	타입	필수	설명	예시
family_id	integer	예	가족 ID(1 고정)	1
situation_id	integer	예	상황 번호(1~12)	4
author	string	예	mom 또는 dad	mom
kind	string	예	voice 또는 text	text
text	string	kind 가 text 일 때	메시지 내용(1~200 자)	민준아, 많이 속상했지
audio_path	string	kind 가 voice 일 때	API-05 가 돌려준 파일 이름	3f2a….m4a

응답은 메시지 전체(id, family_id, situation_id, author, kind, text, audio_path, delivered, created_at, updated_at, deleted)다. 상황과 메시지는 1:1 이다. 지워지지 않은 메시지가 있는 상황에 다시 등록하면 E-409 를 돌려주고, 바꾸려면 API-06 을 쓴다. 녹음은 API-05 로 파일을 먼저 올리고, 받은 audio_path 로 API-04 를 부른다. 오류 응답은 `{"error": "E-409", "message": "이 상황에는 이미 메시지가 있어요"}` 모양이다.

IF-2 디바이스 ↔ 서버(가족이 하나라 family_id 를 받지 않는다)

ID	메서드	주소	기능
API-11	GET	/device/changes?since={시각}	since 이후에 생기거나 고쳐지거나 지워진 메시지를 받는다
API-12	GET	/device/audio/{id}	녹음 파일(m4a)을 내려받는다. 글 메시지는 E-400, 지워진 메시지는 E-404
API-13	POST	/device/ack	저장을 마친 메시지 ID 들을 알린다(`{"message_ids":[1,2]}` → `{"acknowledged_ids":[1,2]}`). 서버는 전달됨으로 바꾼다
API-14	POST	/device/play-logs	재생한 메시지와 시각을 올린다 (`{"message_id":3,"played_at":"…"}`). 새로 저장하면 201, 같은 기록이면 200
API-17	POST	/device/needs	아이가 말한 상황에 부모 메시지가 없을 때 상황 번호와 시각만 올린다. 아이가 한 말은 보내지 않는다(11/11)

API-11 응답

이름	타입	설명	예시
server_time	string	조회 직전 서버 시각. 다음 요청의 since 로 쓴다	2026-11-07T21:39:28
messages[].id	integer	메시지 ID	1
messages[].situation_id	integer	상황 번호	4
messages[].author	string	mom 또는 dad	mom
messages[].kind	string	voice 또는 text	voice
messages[].text	string	글 메시지 내용(voice 면 null)	null
messages[].deleted	boolean	지워진 메시지면 true	false
messages[].updated_at	string	내용이 바뀐 시각	2026-11-07T21:38:24
family	object	자녀 이름과 호칭(11/11 추가). 보드는 이 이름으로 아이를 부른다. 이 요청 때 서버는 스피커 마지막 연결 시각을 남긴다	{"child_name": "민준", …}

디바이스는 앱을 켜면 바로, 이후 30 초마다 다음 순서로 부른다. ① 못 보낸 재생 기록을 API-14 로 올린다. ② API-11 로 변경분을 받는다. ③ 녹음은 API-12 로 받고, 지워진 메시지는 디바이스에서도 지운다. ④ API-13 으로 받은 메시지를 알린다. ⑤ server_time 을 다음 since 로 저장한다. 서버 주소가 바뀌면 since 를 처음으로 되돌려 모두 다시 받는다. 실패하면 since 를 그대로 두고 다음 차례에 다시 하며, 같은 요청을 다시 보내도 안전하다. 운영 환경에서는 이 구간을 MQTT(AWS IoT Core)로 바꿔 서버가 변경을 즉시 알리게 한다(설계).

IF-3 자녀 앱 ↔ 디바이스

디바이스 앱이 WebSocket 서버(포트 8765)를 열고, 자녀 앱이 같은 공유기 안에서 접속한다. 모든 메시지는 type 항목을 가진 JSON 이며, 세부 순서는 신호 규격서를 따른다.

ID	방향	type	함께 보내는 항목	화면
AB-01	앱 → 디바이스	start	없음	5-1 → 5-2
AB-02	앱 → 디바이스	stop	없음. 듣는 중(5-2)에는 말 끝으로 처리, 재생 중(5-5)에는 멈춤(v1.2)	5-2, 5-5
AB-03	디바이스 → 앱	listening	없음	5-2
AB-04	디바이스 → 앱	thinking	text(받아쓴 글)	5-3
AB-05	디바이스 → 앱	empathy	text(실제로 들려준 공감 한마디), has_message, author	5-4
AB-06	디바이스 → 앱	playing	situation_id, message_id, author, kind, text(글 원문, 녹음은 받아쓴 글), seconds(재생 길이, v1.2)	5-5
AB-07	디바이스 → 앱	not_found	text(받아주는 말)	5-6
AB-08	디바이스 → 앱	idle	없음	5-1
AB-09	디바이스 → 앱	error	code(오류 코드)	6-1
AB-10	디바이스 → 앱	safety	text(안전 응답, 상담전화 1388 안내)	5-6 화면 사용

예시는 다음과 같다. 앱이 `{"type": "start"}`를 보내면 디바이스는 `{"type": "empathy", "text": "오늘 상을 받은 날이구나", "has_message": true, "author": "dad"}` 다음에 `{"type": "playing", "situation_id": 11, "message_id": 3, "author": "dad", "kind": "text", "text": "민준아, 상 받은 거 정말 축하해! …"}`를 보낸다. has_message 가 false 이면 앱은 5-4 에서 "이어서 메시지를 들려줄게" 문구를 숨기고 idle 을 기다린다. 개발용으로는 start 에 text 를 넣어 마이크 대신 글을 입력하거나, check 신호로 재생 없이 판단·LLM 결과만 받을 수 있다.

IF-4 디바이스 앱 안 모델 호출

모델 호출은 디바이스 앱의 한 곳(ai_client)에 모았다. 흐름 코드는 아래 네 함수만 부르므로, 모델을 바꿔도 흐름은 바뀌지 않는다.

함수	받는 값	돌려주는 값	실제 모델
stt	16kHz wav	받아쓴 글	SenseVoice
embed	글	384 차원 벡터	MiniLM int8
infer	글	LLM 출력 원문 (`<maum_0>(…)<maum_sep>…`)	Genie(모델이 없거나 꺼져 있으면 가짜 LLM)
tts	글	wav	VITS

LLM 설정(Genie json 경로, 입력 앞머리, 사용 여부)과 서버 주소, 아이 이름은 보드의 설정 파일(config.json)로 빌드 없이 바꾼다.

에러 코드

코드	구간	뜻	처리
E-400	IF-1, IF-2	요청 값이 잘못됨	앱에서 입력을 다시 확인
E-404	IF-1, IF-2	없는 가족, 상황, 메시지, 파일	목록을 다시 조회
E-409	IF-1	가족이 이미 있거나, 메시지가 있는 상황	수정 API 사용
E-413	IF-1	녹음 파일이 10MB 초과	다시 녹음 안내
E-500	IF-1, IF-2	서버 오류	잠시 후 다시 시도
D-001	IF-3	마이크에 말소리가 없음(5 초)	다시 말하기 안내
D-002	IF-3	받아쓰기 결과가 비어 있음	다시 말하기 안내(소리로도 안내)
D-003	IF-3	모델 준비 실패	디바이스 재시작 안내
D-004	IF-3	디바이스에 연결할 수 없음	연결 안 됨 화면(6-1)

4.4 통신 프로토콜 및 형식

구간	프로토콜	형식	비고
IF-1 앱 ↔ 서버	HTTP, RESTful	JSON(녹음은 multipart)	`http://<서버 노트북 IP>:8000`. HTTPS 는 운영 설계
IF-2 디바이스 ↔ 서버	HTTP, RESTful	JSON	온라인일 때만, 30 초 간격
IF-3 자녀 앱 ↔ 디바이스	WebSocket	JSON	같은 공유기 안(포트 8765). 인터넷 없이 동작
IF-4 디바이스 안	함수 호출	글, wav	보드 밖에서는 접근하지 않음

IF-3 에 WebSocket 을 고른 이유는 연결을 유지해 디바이스가 먼저 상태를 알릴 수 있고, 인터넷 없이 공유기만 있으면 동작하며, 블루투스보다 구현이 단순하기 때문이다.

4.5 인증 및 보안

온기에서 보안의 핵심은 아이의 음성과 발화가 디바이스 밖으로 나가지 않는 구조 자체다. 로그인과 통신 암호화(HTTPS)는 설계로 둔다.

항목	이번 구현	실제 서비스에서의 계획(설계)
접근 권한	가족을 하나(family_id = 1)로 고정하고, 같은 공유기 안에서 접속한다	계정 로그인과 토큰으로 가족 단위 접근을 제한한다
앱과 디바이스 연결	디바이스 주소는 앱의 설정값으로 둔다	연결 코드로 짝을 지은 기기만 접속하게 한다
통신 암호화	같은 공유기 안에서 HTTP	HTTPS, 녹음 파일은 비공개 저장소와 기한이 있는 주소
파일 업로드	서버가 파일 이름을 새로 만들고, m4a 형식인지 파일 앞부분까지 확인한다	동일
비밀 정보	DB 주소·비밀번호는 .env 한곳에 두고 GitHub 에 올리지 않는다. 서버는 읽기·쓰기만 되는 전용 DB 계정을 쓴다	동일

항목	이번 구현	실제 서비스에서의 계획(설계)
발화 데이터	음성과 발화 원문을 서버로 보내지 않는다	동일

5. 테스트 계획서 및 결과 보고서

온기의 기능, 성능, 자원 사용을 어떻게 검증할지 계획하고, 수행한 결과를 기록한다. 5.1 은 계획이고 5.2 는 11 월 7 일까지의 결과다.

5.1 테스트 계획서

5.1.1 테스트 목적 및 범위

검증 항목	확인할 내용
기능	부모가 남긴 메시지가 자녀의 말에 맞춰 재생되는 한 사이클이 끝까지 동작하는가
성능	상황 판단(키워드·임베딩, LLM), 잘못 재생 비율, 답 형식, 응답 시간
안정성	해당 없음, 메시지 없음, 연결 끊김, LLM 실패에서 정해진 응답을 하는가. 오래 켜 두어도 동작하는가
자원	추론 중 CPU, 메모리 사용량
오프라인	서버가 없어도 인식, 판단, 재생이 되는가

5.1.2 테스트 전략과 도구

단계	대상	방법·도구
단위	키워드 규칙, LLM 출력 형식, 서버 API	학습·평가 데이터로 키워드 전수 검사, Colab 채점, 서버 /docs 화면
통합	서버 → 디바이스 메시지 전달, 자녀 앱 ↔ 디바이스 신호	실제 서버 코드 + 보드 동기화, 공유기 연결 시험(lan_check), 가짜 보드
시스템	핵심 시나리오 전체	시연 점검(demo_check, 글·파일·마이크), 테스트 결과표 (test_table, 32 문장)
성능	판단 정확도, 응답 시간	보드 LLM 평가(llm_eval, 183 문장), 단계별 시각 로그
음성	받아쓰기 오류	합성 음성 · 잡음 시험(stt_sweep), 팀원의 실제 목소리
안정성	연속 발화, 장시간 구동	발화 반복 입력 후 오류와 속도 변화 기록

부하 테스트는 계획에서 제외한다. 디바이스 한 대가 한 가정에서 한 번에 한 사람의 말을 처리하는 구조이기 때문이다.

테스트 케이스와 현재 상태

ID	구분	내용	기대 결과	결과
TC-01	기능	텍스트 메시지를 등록·수정한다	전달 대기로 표시되고, 디바이스가 받으면 전달됨	통과(앱 등록·수정 → 보드 전달됨, 11/9)
TC-02	기능	녹음 메시지를 등록한다	녹음 파일이 디바이스에 저장된다	통과(앱 녹음 → 보드 재생, 11/9)
TC-03	기능	메시지가 있는 상황에 맞는 말을 건다	공감 한마디 뒤에 부모 메시지가 재생된다	통과(글 4/4, 실제 목소리 4/4)

ID	구분	내용	기대 결과	결과
TC-04	기능	상황과 무관한 말을 건다	받아주는 말이 나온다	통과
TC-05	기능	메시지가 없는 상황에 맞는 말을 건다	부모 메시지 없이 공감 또는 받아주는 말	통과
TC-06	기능	위기 표현이 든 말을 건다	정해진 안전 응답이 나온다	통과
TC-07	기능	텍스트 메시지를 재생한다	온기 목소리로 읽고 앱에 글이 함께 표시된다	통과(재생 화면 글 한 글자씩, 11/12 영상)
TC-08	기능	들은 메시지를 온기기록에서 확인한다	들은 메시지만 표시된다	통과(온기기록, 11/12 영상)
TC-09	통합	말하는 동안 자녀 앱 화면을 확인한다	듣는 중 → 생각하는 중 → 공감 → 재생	통과(11/9 실기, 11/12 영상)
TC-10	온디바이스	서버 없이 TC-03 을 수행한다	동일하게 동작한다	통과(저장된 메시지로 9/9)
TC-11	안정성	앱과 디바이스의 연결을 끊는다	연결 안 됨 안내	앱 시험 통과(가짜 보드)
TC-12	성능	평가 문장으로 LLM 의 상황 판단을 채점한다	90% 이상	Colab 95.6% 통과. 보드는 양자화 모델 수령 후
TC-13	성능	재생 규칙까지 적용해 판단한다	잘못 재생 5% 이하	테스트 결과표 0/6 통과. 평가 183 문장 전체는 수령 후
TC-14	성능	경계 문장, 일반 대화, 관용 표현	잘못 재생 5% 이하	남 이야기·놀이 통과
TC-15	성능	기분 판단	-	v3.7 에서 기분을 상황마다 고정해 상황 판단에 포함
TC-16	성능	말 끝부터 첫 소리까지의 시간	3 초 이내	통과(1.6~1.8 초, 가짜 LLM). 양자화 모델로 다시 측정
TC-17	자원	추론 중 CPU 와 메모리 사용량	보드 실측 기준 이내	수령 후
TC-18	안정성	1 시간 동안 켜 두고 발화를 50 회 반복한다	멈춤 없음, 응답 시간 유지	11/25 전
TC-19	안정성	LLM 이 없는 상태로 TC-03 을 수행한다	키워드·임베딩 결과로 공감과 부모 메시지	통과
TC-20	성능	판단 규칙 A: 키워드가 잡고 LLM 이 일반으로 본 말	키워드대로 대체 공감 + 부모 메시지	통과("시험 완전 망했어")
TC-21	기능	재생 중 "그만 들을래"를 누른다	확인 창이 뜨고, 계속 들으면 끝까지 재생된다	앱 시험 통과(11/12)
TC-22	기능	이름 변경, 메시지 없는 상황, 기록 지우기	스피커가 새 이름으로 부르고, 부모 홈에 필요 카드가 뜨며, 기록이 지워진다	서버·앱 시험 통과(11/11)

5.1.3 테스트 환경

구분	내용
하드웨어	QCS6490 보드, 핀마이크, 미니 스피커, USB 사운드카드, 공유기, 맥(adb·서버)
소프트웨어	디바이스 앱, 오디오 도우미, 서버(실제 코드 또는 가짜 서버), 가짜 보드
도구	보드 로그(adb logcat), 시연 점검·테스트 결과표·LLM 평가·STT 시험·공유기 시험 스크립트, 서버 /docs

5.1.4 테스트 데이터

데이터	용도	규모
평가 문장 v3.7	LLM 판단 정확도(Colab·보드)	183 문장(상황 120, 일반 63)
시연 점검	한 사이클과 재생 규칙	시연 4 + 규칙 5
테스트 결과표	전체 흐름(글·마이크)	32 문장(시연 4, 평가 문장 20, 일반·남 이야기 6, 규칙 A 1, 위기 1)
실제 목소리	받아쓰기를 포함한 전체 흐름	시연 4 + 아이 말투 짧은 말 10
부모 메시지	재생 확인	상황 3·4·10·11 각 1 개

5.1.5 합격 기준

항목	합격 기준
기능(TC-01 ~ TC-11, TC-19, TC-20)	모든 케이스가 기대 결과와 같다
상황 판단(LLM)	90% 이상
잘못 재생 비율	5% 이하
LLM 응답 시간	1 초 이내
전체 응답 시간	3 초 이내
양자화 전후 차이	3%p 이내
오프라인 동작	서버 없이 TC-03 이 통과한다

5.2 테스트 결과 보고서

5.2.1 테스트 개요

항목	내용
테스트 기간	2026 년 9 월 30 일(데이터 진단) ~ 11 월 12 일
대상 버전	학습 데이터 v3.7, LLM v3.7 병합 모델, 저장소 main(디바이스·서버·앱 통합본)
테스트 책임자	공지현(팀장), 파트별 담당자

5.2.2 테스트 결과 요약

보드 단독 한 사이클은 글, 음성 파일, 실제 마이크 모두에서 통과했고, 11 월 9 일 부모 앱부터 보드 재생까지 네 파트를 이은 한 사이클도 실제 마이크로 시연 문장 4/4 를 통과했다. 테스트 결과표 32 문장은 32/32, 첫 응답은 약 1.6~1.8 초다. LLM 은 Colab 에서 95.6%(형식 깨짐 0)로 목표를 넘었고, 보드 양자화 모델 평가가 남아 있다. 앱은 자동 시험 80 개를 모두 통과했다.

5.2.3 항목별 상세 결과

시험	날짜	조건	결과
테스트 결과표 (test_table)	11/7	글 입력, 가짜 LLM, 실제 서버	32/32, 첫 응답 평균 1.6 초, 재생 기록 32 건 서버 저장
실제 목소리 · 시연 문장	11/7	핀마이크, 사운드카드	4/4(띄어쓰기가 틀린 "친 구랑"도 키워드가 받음)
실제 목소리 · 아이 말투 10 문장	11/7	핀마이크	7/10 → 오인식("싸왔어", "다 떴어")을 키워드에 보강. 나머지는 말을 놓치거나 학습 데이터상 일반 대화("엄마 언제 와")
시연 점검 (demo_check)	11/5~7	글 입력	9/9(시연 4, 규칙 A, 일반 대화, 막는 말, 메시지 없는 상황, 위기 표현)
말 끝 판단(--stream)	11/7	조용함을 붙인 음성 파일 6 개	6/6, 말 끝 뒤 0.8 초에 멈춤
응답 시간	11/7	말 끝 → 공감 소리	말 끝 확인 0.8 + STT 0.19~0.24 + 판단·LLM(가짜) 0.3 + TTS 0.27~0.48 = 1.6~1.8 초
소리 크기	11/7	재생 파일 구간별	공감·연결 문장·부모 메시지 차이 1.5dB 이내
서버 동기화	11/7	실제 서버 코드(MySQL)	메시지 4 개·녹음 2 개 수신, 전달됨 표시, 수정이 30 초 안에 반영, 삭제된 메시지가 되살아나지 않음
공유기 연결 (lan_check)	11/7	ipTIME, 보드 랜 + 맥 와이파이	보드 → 서버 닿음, 동기화·재생 기록 통과
LLM 학습 채점	11/5	Colab, 병합 fp16, 평가 183	175/183(95.6%), 상황 113/120, 일반 62/63, 형식 깨짐 0
키워드 규칙	11/7	학습·평가 v3.7	상황 문장 학습 745/745, 평가 120/120
STT 시험(stt_sweep)	11/5	합성 음성, 빠르기 3 × 잡음 2	73/84. 틀린 것은 잡음이 큰 경우
보드 LLM(예제 모델)	11/5~7	마음 AI 예제 모델	LLM 0.65~1.2 초. 답이 우리 형식이 아니어도 규칙 A 로 시연 4/4 유지
네 파트 한 사이클	11/9	부모 앱 녹음 → 서버 → 보드 → 실제 마이크	시연 문장 4/4, 메시지 삭제가 30 초 안에 보드에 반영
응답 시간(실제 마이크)	11/9	말 끝 → 공감 소리 준비, 예제 LLM	STT 0.25~0.4 + 판단·LLM 약 0.3 + TTS 0.27~0.3 = 0.82~1.02 초(말 끝 확인 0.8 초 별도)
녹음 메시지 받아쓰기	11/9	보드 SenseVoice, 31 초 녹음	사람 목소리는 거의 정확(조사 1~2 곳), 합성 음성 임시 녹음은 핵심 단어가 틀림 → 시연 녹음은 실제 목소리로
추가 API	11/11	MariaDB, 기존 DB 에 migrate 적용	API-15~19 와 API-11 family·마지막 연결 통과

시험	날짜	조건	결과
앱 자동 시험	11/12	flutter analyze, flutter test	문제 0, 80/80 통과(재생 중 확인 창, 생각하는 중 0.8 초 포함)
시연 영상 연습본	11/12	윈도우 에뮬레이터 + 맥 서버 + 보드	부모용 66 초, 자녀용 57 초, 실제 장비 41 초

데이터 진단(9 월 30 일~10 월)에서 글자 단위 분류기로 처음 보는 표현의 정확도를 재 보니 초기 데이터(v3)는 32.0%였고, 다시 만든 v3.3 에서 94.5%가 되어 지금의 데이터 작성 방식의 근거가 되었다.

5.2.4 결함 분석 및 조치 사항

번호	결함 내용	심각도	조치
1	앱에서 USB 사운드카드로 소리가 나지 않고 녹음도 안 됨(USB 오디오 오프로드)	높음	root 오디오 도우미(tinyplay·tinycap)로 우회. 실제 소리 확인(완료)
2	지난 녹음 파일을 새 녹음으로 읽어 0.5 초 만에 "말 없음"	높음	녹음 요청 전에 지난 파일을 지움(완료)
3	서버 주소를 바꾸면 이전 서버의 시각 때문에 메시지를 놓칠 수 있음	중간	주소가 바뀌면 처음부터 다시 받음(완료)
4	부모가 지운 메시지가 앱을 다시 켜면 시연용으로 되살아남	중간	서버와 동기화한 뒤에는 시연용 메시지를 채우지 않음(완료)
5	실제 목소리 오인식("싸왔어", "다 떴어", "사웠어")으로 상황을 놓침	중간	두 번 이상 나온 오인식만, 다른 말에 걸리지 않게 범위를 좁혀 키워드에 추가. 데이터 판단 변화 0(완료)
6	일부 시연 문장을 예제 LLM 이 엉뚱하게 답함	중간	판단 규칙 A 와 대체 공감(완료)
7	보드를 다시 켜면 첫 동기화 전까지 재생 기록의 날짜가 틀림(보드 시계가 켜질 때마다 되돌아감)	낮음	서버가 켜져 있으면 1 초 안에 맞춰짐. 시연 전 서버 재생 기록을 정리함
8	공감에 아이가 부른 호칭이 붙음(평가 183 중 3)	낮음	양자화 모델 평가 뒤 보드에서 거를지 결정
9	앱에서 말하기를 누르면 말하지 않아도 "친구와 싸운 날"이 재생됨	높음	보드 시험 모드 기본값을 끔(완료)
10	윈도우 앱이 보드에 닿지 않음(보드와 PC 가 다른 공유기)	높음	맥 중계(ws_relay, 8766)로 USB 하나에 모음(완료)
11	부모 녹음이 보드에서 무음(에뮬레이터 마이크 꺼짐·볼륨 0)	중간	녹음 무음 검사 도구, 녹음 화면 소리 막대·무음 경고(완료)
12	앱을 새로 설치한 직후 LLM 이 첫 실행에 실패	중간	준비 스크립트가 한 번 더 켬(완료)
13	재생 중 "그만 들을래"를 눌러 재생 기록이 남지 않음(11 초 만에 stop)	중간	X 숨김, 멈춤 확인 창, 글 속도를 보드 TTS(초당 약 10 자)에 맞춤(완료)
14	재생 길이(seconds)가 앱에 오지 않음	낮음	보드에 최신 디바이스 앱 설치 여부 확인(다음 실기)
15	같은 삭제를 두 번 받음(초 단위 시각 비교)	낮음	놓치지 않는 쪽이라 결과가 같아 그대로 둠

5.2.5 결론 및 개선 권고

시연에 필요한 기능은 부모 앱부터 디바이스 재생까지 모두 통과했고, 응답 시간은 목표(3 초)의 절반 수준이다. 양자화한 학습 모델의 보드 정확도·시간과 자원·장시간 구동은 최종 시험표에서 확인한다. 시연은 조용한 곳에서 핀마이크를 입 가까이 두고, 시연 문장 "오늘 친구랑 싸웠어"로 진행한다.

6. 배포 및 통합 가이드

개발한 모델과 앱을 QCS6490 디바이스에 올리고 앱, 서버와 연결하는 절차를 정리해, 팀원 누구나 같은 순서로 환경을 다시 만들 수 있게 한다.

6.1 문서 개요

본 문서는 디바이스 앱과 모델을 보드에 올리고, 서버와 앱을 연결하는 실제 절차와 겪은 문제를 다룬다. 각 파트의 README 만 보고 다른 팀원이 같은 결과를 낼 수 있어야 완료로 본다.

6.2 배포 환경 설정

항목	내용
대상 기기	Coretronic COREKOM R6490Q(QCS6490), Android 13 userdebug, adb root 가능, adb 시리얼 75ef63f(USB-C)
주변 장치	USB 사운드카드(보드 USB-A 에 직접), 핀마이크, 미니 스피커(USB 전원 별도), 공유기(보드는 가운데 랜 포트 ↔ 공유기 LAN 포트)
개발 PC	macOS, Android Studio(디바이스 앱 빌드·설치), adb(Android SDK 것만 사용), scrcpy(항상 `--no-audio`), Python 3 + websockets(점검 도구)
서버 PC	Python 3.13, MySQL 8.0.16 이상(개발 26.10), 가상환경 + requirements.txt, `uvicorn main:app --host 0.0.0.0 --port 8000`
LLM 학습	Google Colab T4, 학습·병합 노트북(ai/notebooks)

설정값

항목	위치	값
서버 주소(디바이스)	보드 `/data/local/tmp/ongi/config.json` 의 server_url	USB 개발: `http://127.0.0.1:8000`(adb reverse) / 공유기: `http://<서버 PC IP>:8000`. 빌드 없이 바꾸며 30 초 안에 반영
아이 이름	같은 파일의 child_name	민준
LLM 설정	같은 파일의 llm_config, llm_prefix, use_genie	Genie json 경로, `<|im_start|>`, true/false. 앱을 다시 켜면 적용
서버 주소(앱)	app/lib/config.dart	`http://<서버 PC IP>:8000`
디바이스 주소(앱)	app/lib/config.dart	`ws://<보드 IP>:8765`
DB 접속	server/.env	DB 주소, 계정, 비밀번호, 시간대 +09:00. GitHub 에 올리지 않는다

공유기에서는 주소가 자동으로 배정된다(11/7 보드 192.168.0.2). 고정 IP 를 쓰지 않을 때는 그날 서버 PC 의 IP 를 확인해 server_url 에 넣는다.

6.3 모델 및 추론 엔진 설치

모델	파일	보드 위치
SenseVoice, VITS	sherpa-onnx 용 ONNX 모델(STT 240MB, TTS 77MB, espeak-ng-data). SUDA 앱에서 꺼냄	`/data/local/tmp/ongi/speech/`
silero VAD	silero_vad.onnx	`/data/local/tmp/ongi/speech/vad/`
MiniLM	minilm_int8.onnx, vocab.tsv, 기준 문장 벡터(anchors)	앱 저장소 `files/models/`
Llama-3.2-1B(양자화)	context binary, tokenizer.json, Genie 설정 json, HTP 확장 설정 json	`/data/local/tmp/ongi/llm/<폴더>/`
라이브러리	libGenie, libQnnHtp 등 6 개(SUDA 앱에서 꺼냄), onnxruntime-android 1.17.1(Gradle)	디바이스 앱 안

양자화 모델 교체 절차(빌드 없이)

1. 마음 AI 에서 받은 파일 폴더를 교체 스크립트(`swap_llm.sh <폴더>`)에 넘긴다. 스크립트는 파일을 보드에 넣고, Genie json 안의 경로를 보드 폴더로 고친다.
2. tokenizer 의 새 토큰 4 개 번호(128256~128259)와 Genie 설정의 eos 에 `<maum_end>`가 있는지 점검한다.
3. 설정 파일에 llm_config 와 use_genie 를 적고 앱을 다시 켜, "Genie 준비 완료"와 두 문장 시험 결과를 확인한다.
4. 보드 LLM 평가(`llm_eval.py`, 평가 183 문장)와 시연 점검(`demo_check.py`)을 돌린다. 되돌릴 때는 `swap_llm.sh --back`.

이 절차는 마음 AI 예제 모델로 미리 돌려 확인했다(11/7). 예제 모델은 Genie 가 특수 토큰을 글자 그대로 내어(`<maum_3>()`), 보드의 LLM 답 해석을 그대로 쓸 수 있음을 확인했다.

6.4 온디바이스 시스템 통합 방법

보드 켜는 순서

1. 맥과 보드를 USB-C 로 연결하고 보드 전원을 꽂는다(주황 불).
2. 1~2 분 뒤 초록 불(부팅 완료)이 켜지면 사운드카드를 보드 USB-A 에 꽂는다. 헤드폰 단자에 스피커, 마이크 단자에 핀마이크를 꽂고 스피커 USB 전원을 넣는다.
3. 공유기를 쓸 때는 보드 가운데 랜 포트와 공유기 LAN 포트를 연결한다.
4. 준비 스크립트(`start_board.sh`)를 돌린다. 보드 연결, root, 오디오 도우미, 모델 파일, 앱 다시 켜기, 포트, 준비 완료까지 한 번에 확인한다.
5. 시연 점검(`demo_check.py`)에서 9/9 를 확인한다.

재시작은 전원을 뽑고 10 초 뒤 다시 꽂는다(adb reboot 뒤에는 USB 재인식이 되지 않았다). 사운드카드를 다시 꽂거나 보드를 다시 켜면 준비 스크립트부터 다시 돌린다.

서버 연결

1. 서버 PC 에서 `uvicorn main:app --host 0.0.0.0 --port 8000` 으로 서버를 켠다. Windows 는 방화벽에서 8000 번을 허용한다.

2. 서버 PC 의 IP 를 확인하고 공유기 연결 시험(`lan_check.sh <서버 IP>`)을 돌린다. 보드 → 서버 접속, 서버 주소 변경, 동기화, 한 바퀴와 재생 기록까지 확인한다.
3. 서버 로그에서 API-11 → 12 → 13 → 14 순서를, DB 에서 전달됨 표시를 확인한다.

앱이 없을 때는 서버의 시험 화면(`/docs`)이나 API 호출로 메시지를 등록·수정해 서버 → 디바이스 구간을 확인한다. 시연 전에는 시험 중 쌓인 서버 재생 기록을 지운다.

앱까지 이은 시연 구성

앱은 윈도우 PC 의 안드로이드 에뮬레이터에서, 서버와 보드 도구는 맥에서 돌린다(맥 에뮬레이터는 화면 그리기가 느렸다). 윈도우와 맥은 휴대폰 핫스팟 하나에 함께 연결해 장소의 와이파이 설정(기기 간 차단 등)에 영향받지 않게 하고, 보드는 맥과 USB 하나로 연결해 서버 동기화(adb reverse)와 앱 신호(adb forward + 맥 중계)를 함께 주고받는다.



<그림 6-1> 시연 구성: 윈도우 에뮬레이터 — 휴대폰 핫스팟 — 맥(서버·중계) — USB — 보드

- 맥 창 1(서버): ~/ongi-server/server 에서 uvicorn main:app --host 0.0.0.0 --port 8000
- 맥 창 2(보드): ~/ongi 에서 bash device/tools/start_board.sh. 사운드카드는 보드 부팅이 끝난 뒤 꽂는다
- 맥 창 3(점검): python3 device/tools/demo_check.py
- 맥 창 4(중계): python3 device/tools/ws_relay.py
- 윈도우: C:\src\ongi\app 에서 에뮬레이터(Pixel_8)를 켜고 flutter run. 연결 주소는 앱 설정 > 연결 주소에서 바꾼다
- 튜토리얼을 다시 보려면 앱 설정 > 이 기기 처음 상태로(연결 주소와 서버 메시지는 남는다)

설정 및 구성 매개변수

매개변수	설명	값
키워드·막는 말·위기 표현	situations.json(앱 assets)	12 개 상황, 위기 표현 7 개

매개변수	설명	값
임베딩 임계값	이보다 낮은 유사도는 없음	0.85, 없음과의 차이 0.02
말 끝 판단	말 뒤 조용한 시간 / 말 없음 / 최대 녹음	0.8 초 / 5 초 / 8 초
LLM 기다리는 시간	넘으면 LLM 실패로 처리	8 초
아이 말 길이	LLM 입력으로 자르는 길이	60 자
메시지 확인 간격	디바이스가 서버에 요청하는 간격	30 초, 앱 시작 직후 1 회
오디오 형식	녹음 / 재생	48kHz 모노 → 16kHz / 48kHz 스테레오
녹음 파일	앱 녹음 → 보드 재생	m4a, 최대 10MB

6.5 에러 처리 및 트러블슈팅 가이드

구현하면서 실제로 겪은 문제다.

날짜	증상	원인	해결
11/12	튜토리얼이 다시 나오지 않음	앱을 다시 켜는 것만으로는 첫 설정 기록이 남아 있음	앱 설정 > 이 기기 처음 상태로
11/12	"서버에 연결할 수 없어요"	앱 연결 주소 기본값이 에뮬레이터용(10.0.2.2)	앱 설정에서 맥 IP 입력. 시연 PC 만 기본값을 맥 IP 로(커밋하지 않음)
11/12	시연 DB 스크립트 2 번째 줄 오류	situation 표에 recommended 칸이 없음	칸을 빼고 다시 실행
11/12	온기기록이 비어 있음	재생 11 초 만에 앱에서 stop	끝까지 듣기, 재생 중 확인 창 추가
11/9	부모 녹음이 보드에서 무음	에뮬레이터 마이크 꺼짐·볼륨 0	녹음 무음 검사 (check_audio.sh), 녹음 화면 소리 막대
11/9	윈도우 앱이 보드에 닿지 않음	보드는 기자재 공유기, PC 는 집 공유기	맥 중계(ws_relay, 8766)
11/9	웹 미리보기에서 역할 화면 오류	웹 디버그 컴파일러와 큰 아이콘 묶음	릴리스 모드로 실행
11/7	마이크로 말하기 전에 0.5 초 만에 "말 없음"	지난 녹음 파일을 새 녹음으로 읽음	녹음 요청 전에 지난 파일을 지움
11/7	보드가 공유기 서버에 닿지 않음	보드에 랜선이 없어 네트워크 주소가 없었음	보드 가운데 랜 포트 ↔ 공유기 LAN, 맥은 같은 공유기 와이파이
11/7	모델 교체 스크립트가 tokenizer 를 못 찾음	예제 모델은 json 만 폴더에 있고 파일은 다른 곳에 있음	같은 폴더에 없으면 json 에 적힌 원래 경로를 씀
11/5	STT·TTS 라이브러리 로드 실패(OrtGetApiBase)	onnxruntime 버전이 sherpa-onnx 와 다름	onnxruntime-android 를 1.17.1 로 맞춤
11/5	앱이 오디오 도우미를 " 꺼짐"으로 오판	도우미가 살아 있음 파일을 비우고 쓰는 순간에 앱이 읽음	임시 파일에 쓴 뒤 이름을 바꿈
11/5	첫 발화의 STT 가 3 초	첫 실행 때 모델 준비	앱을 켤 때 STT 를 한 번 미리 돌림 (0.1 초)

날짜	증상	원인	해결
11/5	빌드는 되나 코드가 빠짐	Kotlin 한 줄 뒤에 붙인 `//` 설명이 나머지를 주석으로 만듦	설명은 윗줄에 씀
10/30	앱에서 소리가 안 나고 `write blocked`, 녹음은 멈춤	USB 오디오 오프로드로 Android 경로가 막힘	root 도우미가 tinyplay·tinycap을 씀
10/30	소리가 맥에서 나옴	scrcpy 가 보드 소리를 가져감	항상 `scrcpy --no-audio`
10/30	스피커에서 아무 소리가 없음	스피커 USB 전원이 헐거움	다시 꽂고 미세한 잡음으로 전원 확인
10/30	tinyplay `Error playing sample`	시험 파일이 모노	재생은 48kHz 스테레오로 변환
10/30	adb protocol fault 반복	brew adb 와 Android Studio adb 가 충돌	SDK platform-tools 를 PATH 앞에 둠
10/30	adb reboot 뒤 연결 안 됨	재부팅 후 USB 재인식 실패	전원을 뽑고 10 초 뒤 다시 꽂음
10/26	사운드카드 번호가 바뀜	꽂은 채 부팅하면 USB 가 0 번	부팅 후 꽂기로 통일, 도우미가 카드 번호를 매번 찾음
-	재생 기록 시각이 틀림	보드 시계가 켜질 때마다 2025 년으로 돌아감	서버 시각과의 차이로 보정

로그 확인 및 디버깅 방법

- 보드 로그는 `adb logcat -s ongi` 로 본다. 오디오 도우미 로그는 `/data/local/tmp/ongi/audio.log` 에 있다.
- 단계마다 받아쓴 글, 걸린 키워드, 임베딩 1 순위와 유사도, LLM 답, 적용한 재생 규칙, 단계별 시간("한 바퀴: 말 끝 → 공감 소리 준비 ○○ms")을 남긴다.
- 아이의 발화가 담긴 로그는 디바이스 안에서만 확인하고 서버로 보내지 않는다.

6.6 업데이트 및 유지보수 전략

대상	방식	설명
부모 메시지	자동	온라인일 때 디바이스가 서버에서 추가·수정·삭제를 받아 반영한다
서버 주소, 아이 이름, LLM 설정	설정 파일	config.json 을 바꾸면 빌드 없이 반영된다
키워드, 기준 문장	수동(앱 다시 설치)	situations.json 과 기준 문장 파일을 바꾼다. 바꾸기 전에 학습·평가 데이터의 판단이 바뀌지 않는지 확인한다
LLM	수동(파일 교체)	교체 스크립트로 파일을 바꾸고 보드 평가를 다시 돌린다
디바이스 앱, 모바일 앱, 서버	수동 설치	GitHub 에서 받아 다시 설치한다(디바이스는 Android Studio, 앱은 APK)

모델과 데이터의 무선(OTA) 업데이트는 설계 과제로 둔다.

대상	표기	예시
데이터셋	v + 번호. 바꾼 내용과 이유를 기록	v3.7

대상	표기	예시
LLM 모델	데이터 버전 + 형태	ongi_llama3.2-1b_v3.7_merged
소스 코드	GitHub 저장소 1 개. 파트별 브랜치(app, server, device, ai)에서 작업하고 점검일에 main 에 합친다	-

부록 A. 상황 12 개 대응표

화면 이름, 데이터 번호, 표현 단서를 하나로 맞춘 기준표다. 부모가 고르는 상황과 AI 가 찾는 상황은 이 표의 12 개와 같아야 한다. 추천은 "추천 10 개 채우기"에 들어가는 상황이고, 시연은 11/26 시연 문장에 쓰는 상황이다.

번호	화면 이름	분류 / 허용 기분(기본)	표현 단서(사건 + 마음)	추천	시연
1	학교 가기 싫은 날	학교·공부 / 2, 3, 4 (기본 4)	학교·등교·개학 + 가기 싫다·쉬고 싶다·무섭다	●	
2	실수해서 혼났을 때	학교·공부 / 1, 2, 3 (기본 1)	(내가) 혼났다·야단·꾸중 + 속상·억울·무서움	●	
3	시험을 망친 날	학교·공부 / 1, 2, 3, 4 (기본 1)	시험·대회 결과 + 망쳤다·떨어졌다·점수 낮다	●	●
4	친구와 싸운 날	친구 / 1, 2 (기본 1)	친구·형제와(랑·하고) + 싸웠다·다퉜다·말다툼	●	●
5	친구가 날 싫어하는 것 같을 때	친구 / 1, 2, 3 (기본 1)	애들·친구들이 나를 + 빼고·무시·따돌림·뒷담	●	
6	남들과 비교될 때	친구 / 1, 2 (기본 1)	다들·친구·인스타 + 부럽다·비교·나만 없다		
7	울고 싶은 날	마음 / 1, 2 (기본 1)	(내가) 울고 싶다·울었다·눈물·울컥	●	
8	아무것도 하기 싫은 날	마음 / 2, 4 (기본 4)	(내가) 귀찮다·하기 싫다·지쳤다·힘이 없다	●	
9	잠이 안 오는 밤	마음 / 2, 3 (기본 3)	밤·잠 + 잠이 안 온다·못 자겠다·뒤척인다	●	
10	부모님이 보고 싶을 때	가족·특별한 날 / 5 (기본 5)	엄마·아빠·부모님 + 보고 싶다·그립다·목소리 듣고 싶다	●	●
11	자랑하고 싶은 날	가족·특별한 날 / 6 (기본 6)	상·일등·백 점·반장·합격·칭찬 + (내가) 받았다·했다	●	●
12	생일(내 생일만)	가족·특별한 날 / 5, 6 (기본 6)	(내) 생일 + 오늘·내일·축하·케이크·선물		

기분 번호는 1 슬픔·속상함, 2 화남·억울함, 3 불안·긴장, 4 지침·무기력, 5 외로움·그리움, 6 기쁨·뿌듯함, -1 말하지 않음이다. 24 개 상황에서 빠진 12 개는 정확도를 확보한 뒤 추가를 검토한다.